IT PROJECT REPORT ON
STUDENT INDUSTRIAL WORK EXPERIENCE SCHEME (SIWES)
UNDERTAKEN AT

ACADEVA LIMITED

BY

JOSEPH COSMOS ANIEBIET
22/SC/CO/1301

DEPARTMENT OF COMPUTER SCIENCE,
FACULTY OF COMPUTING,
UNIVERSITY OF UYO, UYO.

SUBMITTED TO

DEPARTMENT OF COMPUTER SCIENCE,
FACULTY OF COMPUTING,
UNIVERSITY OF UYO, UYO.

IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF A BACHELOR OF SCIENCE (B.Sc.) DEGREE IN COMPUTER SCIENCE.

OCTOBER, 2026.

IT PROJECT ON

ARACARE HEALTH OS:

AN OFFLINE-FIRST HOSPITAL OPERATING SYSTEM WITH INTELLIGENT PATIENT QUEUING, ELECTRONIC HEALTH RECORDS AND A LOCAL AI VOICE SCRIBE

BY

JOSEPH COSMOS ANIEBIET

22/SC/CO/1301

DEDICATION

This work is dedicated to Almighty God, the source of all wisdom, knowledge and understanding, who kept me throughout the period of my industrial training;

and to my beloved parents, for their love, prayers and sacrifices, which made my education possible.

ACKNOWLEDGEMENT

I give all glory to Almighty God for the gift of life, good health and the grace to complete my Student Industrial Work Experience Scheme (SIWES) and this report.

I sincerely thank the management and staff of **Acadeva Limited** for accepting me as an intern and for creating an environment where I could learn by doing real work. I am especially grateful to my industry-based supervisor, **Mr. John Henry**, whose guidance, patience and honest feedback shaped every project I worked on, from the phishing detection model to the AI content tools. I also thank the engineering, machine learning and design teams, and the PY50 trainees, for the knowledge they shared with me and the questions that made me understand my own work better.

My gratitude goes to the Head of Department and all the lecturers of the **Department of Computer Science, Faculty of Computing, University of Uyo**, for the academic foundation that made this training meaningful, and to the departmental SIWES coordinator and my institution-based supervisor for their supervision and support. I also acknowledge the **Industrial Training Fund (ITF)** for sustaining the SIWES programme.

Finally, I am deeply thankful to my parents, siblings, friends and coursemates for their encouragement, prayers and support throughout this period. May God bless you all.

ABSTRACT

This report documents the six-month Student Industrial Work Experience Scheme (SIWES) undertaken at Acadeva Limited, an education technology company in Akwa Ibom State, Nigeria, between May and October 2026, and the IT project developed from the skills acquired. During the placement I worked as an AI/ML intern in the Data Science, Machine Learning and Web Development tracks. I built a phishing detection system that merges an email dataset and a URL dataset, extracts TF-IDF features, trains and ranks six candidate classifiers, strengthens the model with generated typosquatting examples, and serves predictions through a tested, rate-limited FastAPI service with a safe champion/challenger retraining flow. I also contributed to four generative-AI content tools (Acadeva Magic, the Book Topic Ranker, the Content Categorizer and the Quithn quiz generator) through prompt engineering, bug fixing and a shared helper refactor, and built a content analytics view. The data science workflow learned is demonstrated with three beginner projects: Titanic survival prediction (82.2% cross-validated accuracy), California house price prediction ($R^2 = 0.81$) and Iris flower classification (97.8% accuracy).

The IT project, **AraCare Health OS**, applies these skills to a real problem in Nigerian hospitals: long, disorderly outpatient queues and paper records. AraCare is an offline-first hospital operating system built with FastAPI, SQLAlchemy and React. Its first release covers patient registration with QR-coded receipts, a queue engine that uses weighted fair queuing with aging so urgent patients are seen first without anyone being starved, a waiting-room TV display, an electronic health record with vital-sign flags, ICD-10 diagnoses and a server-side allergy hard-block, and an AI voice scribe that runs fully on the hospital's own computer using Whisper and a local language model, with grounding rules that remove any value the clinician did not say. All 47 automated tests pass. An analysis of six weeks of system records shows that the priority rules work as designed: in the doctor queue, median waits were about 2 minutes for emergencies, 24 minutes for high-priority patients and 57 minutes for normal patients, while arrivals peak between 08:00 and 11:00.

Keywords: SIWES, data science workflow, machine learning, phishing detection, hospital information system, queue management, electronic health records, speech recognition, FastAPI, React.

TABLE OF CONTENTS

Dedication ..... **iii**
Acknowledgement ..... **iv**
Abstract ..... **v**
List of Figures ..... **viii**
List of Tables ..... **x**
Chapter 1: Introduction to SIWES Placement **1**
1.1 Background of the SIWES Programme 1
1.2 About Acadeva 2
1.3 History, Vision, Mission and Core Values 3
1.4 Duration and Scope of Internship 4
1.5 Conclusion 6
Chapter 2: Practical Skills Acquired During SIWES **7**
2.1 Data Collection and Management 7
2.4 Programming and Software Development 8
2.5 Exploratory Data Analysis 10
2.6 Machine Learning Concepts and Modelling 11
2.7 The Data Science Workflow 15
2.8 Beginner Data Science Projects 17
2.8.1 Project 1: Titanic Survival Prediction 17
2.8.2 Project 2: House Price Prediction 19
2.8.3 Project 3: Iris Flower Classification 20
2.9 Other Technical and Professional Skills 22
2.10 Conclusion 22
Chapter 3: IT Project on AraCare Health OS **24**
3.1 Project Overview 24
3.2 Background of the Study 25
3.3 Problem Statement 26
3.4 Aim and Objectives 26
3.5 Scope and Significance 27
3.6 Methodology and Tools 27
3.7 System Architecture 28
3.8 Design and Implementation of the Core Modules 31
3.8.1 Patient Registration and Check-in 31
3.8.2 The Queue Engine 31
3.8.3 Electronic Health Record and Clinical Safety 33

3.8.4 The AI Voice Scribe....35
3.8.5 Events, Security and Production Safety....36
3.9 System Walkthrough: User Interface....37
3.9.1 Sign In and Account Security....37
3.9.2 Waiting-Room TV Display (Triage and Doctor Queues)....37
3.9.3 Reception: Finding, Registering and Checking In Patients....38
3.9.4 Nurse Triage....40
3.9.5 Doctor Consultation....41
3.9.6 Administration....44
3.10 Data Analysis of AraCare Records (Python Notebook)....44
3.11 Testing and Validation....49
3.12 Deployment....50
3.13 Challenges....50
3.14 Limitations and Future Work....51
3.15 Conclusion....52
References....53
Appendices....55

LIST OF FIGURES

Figure 1.1: Simplified organisational structure of Acadeva and the position of SIWES interns.....3
Figure 1.2: Timeline of activities across the 24-week placement (from the SIWES logbook).....5
Figure 2.1: FastAPI service for the phishing model with Pydantic validation, logging and rate limiting.....8
Figure 2.2: Examples of the pytest suite written for the /predict endpoint.....9
Figure 2.3: The shared JavaScript helper used by Acadeva Magic, the Ranker, the Categorizer and Quithn.....10
Figure 2.4: Six candidate pipelines sharing one TF-IDF front end, and the stratified split.....12
Figure 2.5: Phishing detection model: training and serving pipeline (logbook sketch, weeks 3 to 7).....12
Figure 2.6: Generating typosquatting and brand-spoofing URLs as hard training examples.....13
Figure 2.7: Champion/challenger promotion: the production model is only replaced by a better one.....14
Figure 2.8: Acadeva EdTech AI content tools: the shared prompt-pipeline pattern (logbook sketch,.....15
Figure 2.9: The data science workflow: eight steps in a cycle, with feedback loops.....16
Figure 2.10: Titanic EDA: survival by sex and by class, and age distribution by outcome.....18
Figure 2.11: Random Forest confusion matrix and feature importance for the Titanic model.....18
Figure 2.12: California housing EDA: value by location (point size = population) and income vs. value. 19
Figure 2.13: Random Forest predicted vs. actual house values (points near the red line are accurate).....20
Figure 2.14: Pair plot of the four iris measurements, coloured by species.....21
Figure 2.15: Choosing k for K-Nearest Neighbours, and the confusion matrix of the final model.....21
Figure 3.1: System architecture of AraCare Health OS.....29
Figure 3.2: The data spine: Patient, Visit, Journey, JourneyStep and QueueTicket, with clinical.....30
Figure 3.3: The patient journey and the domain events that drive it.....30
Figure 3.4: Queue ticket state machine.....32
Figure 3.5: Initial priority rules (queues/service.py).....32
Figure 3.6: Aging, ordering and weighted fair queuing in pick_next() (queues/service.py).....33
Figure 3.7: Vital-sign flagging rules (ehr/clinical_rules.py).....34
Figure 3.8: Drug-class allergy matching (ehr/clinical_rules.py).....35
Figure 3.9: AI voice scribe pipeline: from speech to a signed note.....35
Figure 3.10: Grounding: removing numbers the clinician never said (scribe/structure.py).....36
Figure 3.11: Sign-in page (left) and the message shown after a wrong password (right).....37
Figure 3.12: Waiting-room TV display for the triage queue.....38
Figure 3.13: Waiting-room TV display for the doctor (OPD) queue.....38
Figure 3.14: Patient profile with the check-in form (left) and the new patient registration form.....39
Figure 3.15: Check-in receipt with QR code and the triage ticket number TRI-031.....39
Figure 3.16: Live queue overview for triage and the doctor queue.....40
Figure 3.17: Nurse triage dashboard with TRI-025 promoted by aging (left), and a patient called with....40
Figure 3.18: Voice note with recorded consent (left) and the vital signs filled in from it, for the.....41
Figure 3.19: Triage saved, with abnormal vital signs flagged.....41

Figure 3.20: Doctor dashboard with the doctor queue (left) and the patient chart opened for.....42
Figure 3.21: Clinical note from the "Uncomplicated malaria" template (left) and a SOAP note drafted... 42
Figure 3.22: ICD-10 diagnosis search (left) and laboratory orders and prescriptions (right).....43
Figure 3.23: Allergy safety block for a penicillin-allergic patient (left) and the signed visit with.....43
Figure 3.24: Staff management.....44
Figure 3.25: Registered patients by age band and sex, and visits by payment track.....45
Figure 3.26: Check-ins by hour of day and visits per day by weekday.....45
Figure 3.27: Waiting time before being called, by priority tier and queue.....46
Figure 3.28: Total time from joining the triage queue to the end of the consultation.....47
Figure 3.29: Abnormal-vital flags raised at triage, and correlation between vital signs.....47
Figure 3.30: Top 10 primary diagnoses by ICD-10 code.....48
Figure 3.31: Signed notes per week by source, and scribe sessions by role.....48
Figure 3.32: Test run: all 47 backend tests pass.....50

LIST OF TABLES

Table 1.1: Acadeva at a glance..... 3
Table 1.2: Summary of the SIWES placement..... 4
Table 2.1: Classification metrics used to evaluate and rank the phishing models (TP = true..... 12
Table 2.2: Mapping the data science workflow to the phishing detection project..... 17
Table 2.3: Titanic results..... 18
Table 2.4: House price results on the 20% test set..... 19
Table 2.5: Summary of the three beginner data science projects..... 22
Table 3.1: AraCare Health OS at a glance..... 24
Table 3.2: Technology stack of AraCare Health OS..... 27
Table 3.3: Domain events: the only coupling between modules..... 29
Table 3.4: Waiting time by queue and priority tier..... 46
Table 3.5: Summary of findings from the AraCare data analysis..... 48
Table 3.6: Automated test suite of the AraCare backend..... 49
Table A.7: AraCare API surface..... 55

CHAPTER ONE

INTRODUCTION TO SIWES PLACEMENT

INTRODUCTION

This chapter gives an overview of the Student Industrial Work Experience Scheme (SIWES) and introduces the organisation where I completed my industrial training, **Acadeva Limited**. It explains the purpose of SIWES, describes the background and profile of Acadeva, including its history, vision, mission and core values, and outlines the duration and scope of my internship. Together these sections provide the context for the practical skills presented in Chapter Two and the IT project presented in Chapter Three.

1.1 BACKGROUND OF THE SIWES PROGRAMME

The Students Industrial Work Experience Scheme (SIWES) is a skills training programme designed to bridge the gap between the theory learned in the classroom and the practice of industry. It was established by the Industrial Training Fund (ITF) in 1973, following the observation that many graduates of Nigerian tertiary institutions lacked the practical skills that employers needed. Today it is a compulsory part of degree programmes in engineering, technology, science and related disciplines, and is jointly managed by the ITF, the National Universities Commission (NUC), the participating institutions and the employers who host the students.

The scheme places students in real work environments where they apply what they have learned to real problems, under the guidance of experienced professionals. For students of Computer Science, this means working with the tools, codebases, data and development practices that are used in industry today: version control, code review, automated testing, cloud and API services, data pipelines and machine learning systems. It also exposes students to the non-technical side of professional life: deadlines, teamwork, communication with non-technical colleagues, documentation and accountability.

The specific objectives of SIWES, as stated by the Industrial Training Fund, are to:

- provide an avenue for students in higher institutions to acquire industrial skills and experience in their course of study;

- prepare students for the work situation they are likely to meet after graduation;
- expose students to work methods and techniques in handling equipment and machinery that may not be available in their institutions;
- make the transition from school to the world of work easier, and enhance students' contacts for later job placement;
- provide students with an opportunity to apply their theoretical knowledge to real work situations, thereby bridging the gap between theory and practice; and
- enlist and strengthen employers' involvement in the entire educational process of preparing graduates for employment in industry.

The programme typically lasts six months for university students. At the end, the student submits a logbook signed by the industry-based supervisor and a technical report, and is assessed by the institution. This report is part of that requirement.

1.2 ABOUT ACADEVA

Acadeva Limited is a Nigerian education technology company that collects and organises study materials, past questions and PDFs from Nigerian universities by school, department and course. Students use Acadeva to find what they need to study, download it for offline use, share materials with classmates, and keep a shared academic schedule with reminders. Study tools such as quizzes and summaries are built on top of the materials to help students learn from them, and these tools are powered by machine learning and large language models.

Acadeva is available on Android, iOS, iPadOS, macOS, Windows and Linux, and is registered with the Corporate Affairs Commission (CAC). It operates in Akwa Ibom State. The company has a small core team working across engineering, machine learning and design, supported by a wider team of about nine people in development, design and marketing. Students on placement are attached to the Data Science, Machine Learning and Web Development tracks, where they work under the supervision of **Mr. John Henry**. Figure 1.1 shows a simplified view of how the company is organised and where SIWES interns fit.

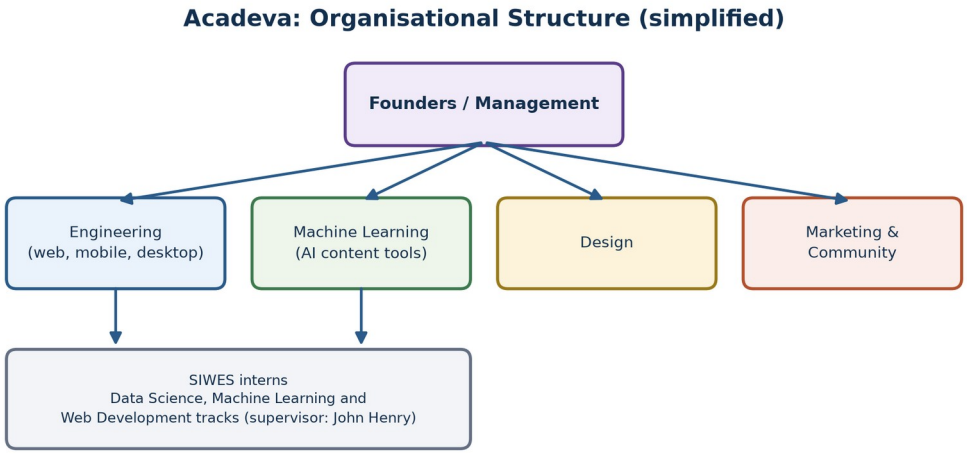


Figure 1.1: *Simplified organisational structure of Acadeva and the position of SIWES interns.*

Table 1.1: *Acadeva at a glance.*

Item	Details
Company	Acadeva Limited (registered with the Corporate Affairs Commission)
Sector	Education technology (EdTech)
Location	Akwa Ibom State, Nigeria
Products	Study-material library, offline downloads, sharing, academic schedule with reminders, AI study tools (quizzes, summaries, metadata generation)
Platforms	Android, iOS, iPadOS, macOS, Windows, Linux
Team	Small core team (engineering, machine learning, design) plus about nine people in development, design and marketing
Intern tracks	Data Science, Machine Learning, Web Development

1.3 HISTORY, VISION, MISSION AND CORE VALUES

Organisational History

Acadeva began as a prototype that won first and second place at a campus technology competition in November 2024. The Android app launched on 20 June 2025, and more than 2,000 students signed up within the first six months. In July 2025 Acadeva was First Runner-Up at the DevEast Entrepreneurship Week at the Federal University of Technology, Owerri (FUTO), and received startup funding.

Since launch, Acadeva has grown through partnerships with student and academic communities. Its library has grown past 1,000 shared books, and it has hosted more than 20 large quiz events. It has since added iOS and iPadOS apps and desktop apps for Windows, macOS and Linux, and

introduced paid subscriptions. Throughout this growth the company has relied on automation, including machine learning, to keep a small team productive: for example, uploaded books are tagged and described by AI tools instead of by hand.

Vision Statement

To make quality academic resources accessible to every student and create a more connected way for students to learn together.

Mission Statement

To make study resources accessible and learning more effective by building collaborative and innovative tools designed around students' real academic needs.

Core Values

- **Accessibility:** study materials should be available to every student, including offline and on low-cost devices.
- **Collaboration:** students learn better together, so sharing and group features are central to the product.
- **Innovation:** new technology, including AI, is used where it genuinely helps students learn.
- **Student-Centricity:** features are built around students' real academic needs, not around technology for its own sake.
- **Integrity:** honesty in how materials are sourced and shared, and in how student data is handled.

1.4 DURATION AND SCOPE OF INTERNSHIP

My SIWES placement at Acadeva ran for six months (24 weeks), from May to October 2026, in the role of **AI/ML Intern**. The placement was structured to give practical exposure to how a student-focused software product is built, maintained and improved. I participated in activities in the Data Science, Machine Learning and Web Development tracks under the supervision of Mr. John Henry.

Table 1.2: *Summary of the SIWES placement.*

Item	Details
Student	Joseph Cosmos Aniebiet (22/SC/CO/1301)

Organisation	Acadeva Limited, Akwa Ibom State
Role	AI/ML Intern (Data Science, Machine Learning and Web Development tracks)
Period	11 May 2026 to 23 October 2026 (24 weeks)
Industry-based supervisor	Mr. John Henry
Main projects	Phishing detection model and API; Acadeva Magic, Book Topic Ranker, Content Categorizer and Quithn (generative-AI content tools); content analytics view

Work was organised in phases, as recorded week by week in my logbook and summarised in Figure 1.2. The first half of the placement focused on building and serving a phishing detection model from scratch. The middle phase moved to the generative-AI content tools that power Acadeva's study features. The final phase was about making both systems production-ready: monitoring, automated tests, performance, security, documentation, integration testing and handover.

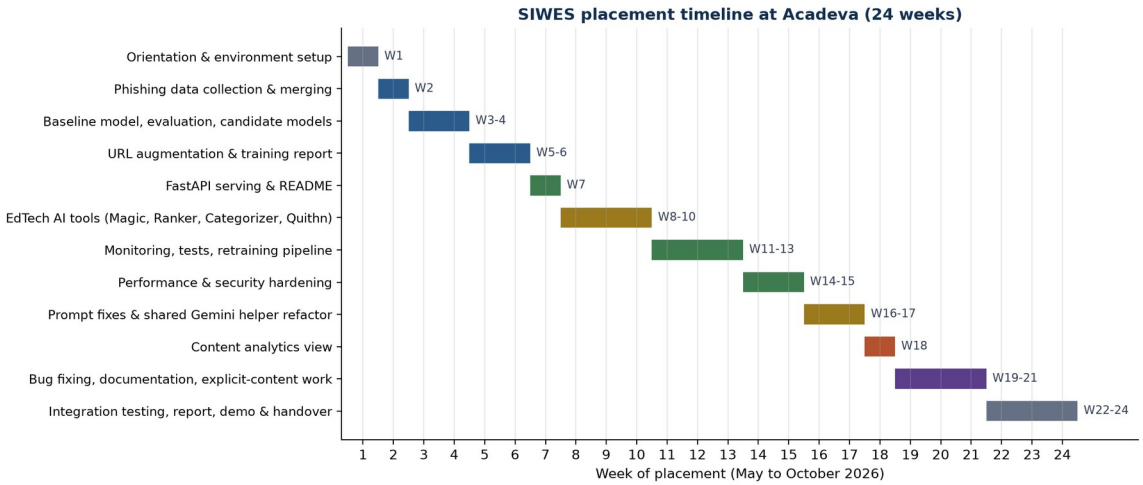


Figure 1.2: *Timeline of activities across the 24-week placement (from the SIWES logbook).*

Within this period I developed practical skills in the following areas:

- **Data collection and management:** sourcing, merging and labelling datasets; tagging study materials by school, department and course.
- **Data cleaning and analysis:** label normalisation, de-duplication, PDF text extraction and review of logged predictions.
- **Data visualisation and exploratory analysis:** class balance, per-source metrics and content analytics with pandas and Matplotlib.

- **Programming and software development:** Python, FastAPI, Pydantic, pytest, JavaScript/Node.js, Git and documentation.
- **Machine learning:** TF-IDF features, linear classifiers, model calibration, evaluation metrics, data augmentation and safe retraining.
- **Generative AI:** prompt engineering, few-shot examples and structured JSON output with the Gemini API.
- **Professional skills:** teamwork, code review, knowledge sharing with PY50 trainees, presentations and handover.

1.5 CONCLUSION

This chapter has established the context of my SIWES placement at Acadeva Limited, a young but fast-growing Nigerian EdTech company whose product depends on data and machine learning. It introduced the purpose of SIWES, the organisation's history, vision, mission and values, and the scope of my internship. The next chapter presents the practical skills and competencies I acquired, including the data science workflow and three beginner data science projects.

CHAPTER TWO

PRACTICAL SKILLS ACQUIRED DURING SIWES

INTRODUCTION

The purpose of this chapter is to give a structured account of the practical skills I acquired during the placement. Following the department's template, each section describes the concept and why it matters, the practical activities I performed, the tools I used, and what I learned. Most of the examples come from the two streams of work recorded in my logbook: the **phishing detection system** (weeks 1 to 7 and 11 to 15) and the **Acadeva generative-AI content tools** (weeks 8 to 10 and 16 to 21). The chapter then presents the data science workflow that ties these activities together, and three beginner data science projects that demonstrate the workflow end to end.

2.1 DATA COLLECTION AND MANAGEMENT

Every data science or machine learning project starts with data, and the quality of the final model can never be better than the quality of the data it learned from. Data collection means finding, obtaining and recording the data needed to answer a question; data management means storing it in a consistent structure, with clear labels and documented sources, so that anyone on the team can use it and reproduce the results.

When Acadeva asked for an in-house model to flag phishing emails and URLs, my first task was to research public datasets. I shortlisted and used two: `phishing_email.csv`, which contains the combined text of emails with a label, and the **PhiUSIIL Phishing URL Dataset**, which contains URLs with a label. The two datasets used different column names and, importantly, different label conventions, so I wrote a loader that renames the columns, normalises the labels so that 1 always means phishing and 0 always means legitimate, adds a `source` column (email or URL), and merges both into one table. Keeping the source column turned out to be valuable later, because it allowed metrics to be broken down per source.

- **Practical activity:** built the content analytics view (category distribution and relevance-score distribution).
- **Tools/technologies used:** pandas, Matplotlib, Seaborn.

- **Skills acquired:** choosing chart types, gathering requirements from non-technical users, presenting findings.

2.4 PROGRAMMING AND SOFTWARE DEVELOPMENT

A model is only useful if other software can use it. A large part of my placement was therefore software engineering: turning a training script into a service with a clear API, tests, documentation, and protection against misuse.

Serving the model with FastAPI

I scaffolded the serving application with **FastAPI**, replacing an earlier plan to use Flask, because FastAPI validates requests automatically with Pydantic and generates interactive documentation at `/docs`. I built a `/health` endpoint and a `/predict` endpoint, with the model loading code in `utils/model_utils.py` (Figure 2.1). Two later improvements came from measuring rather than guessing. Profiling in week 14 showed that the vectoriser was being rebuilt on every request, so I moved model loading to start-up, which gave a clear response-time improvement in simple load tests. In week 15 I tightened the request schema with maximum length and required-field rules, added rate-limiting middleware, and checked that error responses never leak internal stack traces.

phishing-detector/main.py - FastAPI /predict endpoint

```python
app = FastAPI(title="Acadeva Phishing Detector")
model = load_model("saved_models/phishing_model.joblib")   # loaded ONCE at startup, not per request

class PredictRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=20_000)
    source: Literal["email", "url"] = "email"

class PredictResponse(BaseModel):
    label: Literal["phishing", "legitimate"]
    score: float
    details: dict

@app.get("/health")
def health():
    return {"status": "ok", "model": model.name}

@app.post("/predict", response_model=PredictResponse)
@limiter.limit("60/minute")
def predict(req: PredictRequest, request: Request):
    score = float(model.predict_proba([req.text])[0][1])
    log.info("predict source=%s score=%.3f", req.source, score)       # reviewed later for errors
    return PredictResponse(label="phishing" if score >= 0.5 else "legitimate",
                           score=round(score, 4), details={"source": req.source})
```

Figure 2.1: *FastAPI service for the phishing model with Pydantic validation, logging and rate limiting.*

Automated testing

In week 12 I set up **pytest** for the project. The tests cover model loading, phishing and legitimate inputs, and edge cases such as empty text, very long text and non-ASCII characters (Figure 2.2). The new tests immediately uncovered two small bugs in the response schema, which I fixed. I also added a local test-runner script so the suite runs before each commit.

phishing-detector/tests/test_predict.py - pytest

```python
client = TestClient(app)

def test_flags_obvious_phishing_url():
    r = client.post("/predict", json={"text": "http://p4ypal.com/verify-account", "source": "url"})
    assert r.status_code == 200
    assert r.json()["label"] == "phishing"

def test_accepts_legitimate_url():
    r = client.post("/predict", json={"text": "https://www.paypal.com", "source": "url"})
    assert r.json()["label"] == "legitimate"

@pytest.mark.parametrize("text", ["", "x" * 20_001])
def test_rejects_empty_and_oversized_input(text):
    assert client.post("/predict", json={"text": text}).status_code == 422

def test_handles_non_ascii():
    r = client.post("/predict", json={"text": "Ọ̀gá, your account dey locked. Click here"})
    assert 0.0 <= r.json()["score"] <= 1.0
```

Figure 2.2: *Examples of the pytest suite written for the /predict endpoint.*

Shared helper for the AI tools

Acadeva's four AI content tools each repeated the same boilerplate for calling the Gemini API. In weeks 16 and 17 I proposed and carried out a refactor that moved model initialisation, generation settings, JSON output and error handling into one shared helper (Figure 2.3), then migrated all four tools to it and verified that their outputs were unchanged. This made the AI layer much easier to maintain, and it later made it easy to find and fix a conflict between two services that shared the same API rate limit.

acadeva-ai/lib/gemini.js - shared helper for the four AI tools

```
// One place for model setup, JSON output and error handling. Magic, the Topic Ranker,
// the Categorizer and Quithn only supply their own system prompt and output schema.
import { GoogleGenerativeAI } from "@google/generative-ai";

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

export async function runTool({ systemPrompt, input, schema, model = "gemini-2.0-flash" }) {
  const m = genAI.getGenerativeModel({
    model,
    systemInstruction: systemPrompt,
    generationConfig: { responseMimeType: "application/json", responseSchema: schema, temperature: 0.2 },
  });
  try {
    const res = await m.generateContent(input);
    return { ok: true, data: JSON.parse(res.response.text()) };
  } catch (err) {
    console.error(`[${model}] tool call failed:`, err.message);
    return { ok: false, error: "The AI service is busy. Please try again." };
  }
}
```

Figure 2.3: *The shared JavaScript helper used by Acadeva Magic, the Ranker, the Categorizer and Quithn.*

- **Practical activity:** FastAPI service, request logging, pytest suite, performance fix, input validation and rate limiting, shared Gemini helper, README and contribution docs.
- **Tools/technologies used:** Python, FastAPI, Pydantic, Uvicorn, pytest, joblib, JavaScript/Node.js, Gemini API, Git, Swagger UI.
- **Skills acquired:** API design, testing, profiling, secure input handling, refactoring, technical writing.

2.5 EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) means getting to know a dataset before modelling it: its size, types, missing values, distributions and relationships. EDA prevents expensive mistakes, such as training a classifier on data that is 95% one class and then celebrating 95% accuracy.

For the phishing data I examined the class balance of each source, the length of texts, and the most frequent tokens in each class. EDA also shaped evaluation: I added a **per-source metrics breakdown** (email vs. URL vs. augmented) to the training report, because an overall score hid the fact that the first models were much weaker on disguised URLs than on emails. For the Book Topic Ranker, I compared the distributions of relevance scores across sample PDFs to flag books whose scores were inconsistent. All EDA was done in Jupyter Notebooks with pandas, Matplotlib and Seaborn, which keep code, output and explanation together in one document.

- **Practical activity:** class-balance checks, text-length and token analysis, per-source metrics, score-distribution comparison.
- **Tools/technologies used:** Jupyter Notebook, pandas, Matplotlib, Seaborn.

- **Skills acquired:** forming questions about data, spotting imbalance and hidden weaknesses, using EDA to design evaluation.

2.6 MACHINE LEARNING CONCEPTS AND MODELLING

Machine learning is the practice of building models that learn patterns from data instead of being explicitly programmed. My work covered two kinds: **supervised classification** for phishing detection, and **generative AI** with large language models for the EdTech tools.

Phishing detection: from baseline to six ranked candidates

Text must be converted to numbers before a classifier can use it. I used **TF-IDF** (term frequency-inverse document frequency), which gives each word or word pair a weight that is high when it is frequent in one document but rare across all documents. I used uni-grams and bi-grams, and added character-level features with a configurable cap after the full dataset caused a memory error. The first model was a Logistic Regression baseline evaluated on a **stratified** train/test split, so both sets keep the same proportion of phishing examples.

I then extended the training script with a `get_model_candidates()` function that returns six TF-IDF pipelines (Figure 2.4): Logistic Regression, Linear SVC, a Linear SVC wrapped in `CalibratedClassifierCV` (so the API can return a meaningful confidence score, which a plain SVM does not provide), Multinomial Naive Bayes, an SGD classifier and a Passive-Aggressive classifier. A standard loop fits, times and evaluates every candidate on the same split, ranks them by a selectable metric with tie-breakers, writes everything to `training_report.json`, and saves the winner with joblib. Figure 2.5 shows the whole pipeline.

phishing-detector/train.py - candidate models

```python
def get_model_candidates(max_features: int = 200_000) -> dict[str, Pipeline]:
    """Every candidate shares the same TF-IDF front end, so results are directly comparable."""
    def tfidf():
        return TfidfVectorizer(ngram_range=(1, 2), max_features=max_features,
                               min_df=2, sublinear_tf=True)
    return {
        "logistic_regression": Pipeline([("tfidf", tfidf()), ("clf", LogisticRegression(max_iter=2000))]),
        "linear_svc": Pipeline([("tfidf", tfidf()), ("clf", LinearSVC())]),
        "calibrated_svc": Pipeline([("tfidf", tfidf()),
                                    ("clf", CalibratedClassifierCV(LinearSVC(), cv=3))]),
        "multinomial_nb": Pipeline([("tfidf", tfidf()), ("clf", MultinomialNB(alpha=0.1))]),
        "sgd": Pipeline([("tfidf", tfidf()), ("clf", SGDClassifier(loss="modified_huber"))]),
        "passive_aggressive": Pipeline([("tfidf", tfidf()), ("clf", PassiveAggressiveClassifier())]),
    }


X_train, X_test, y_train, y_test = train_test_split(
    data["text"], data["label"], test_size=0.2, stratify=data["label"], random_state=42)
```

Figure 2.4: *Six candidate pipelines sharing one TF-IDF front end, and the stratified split.*

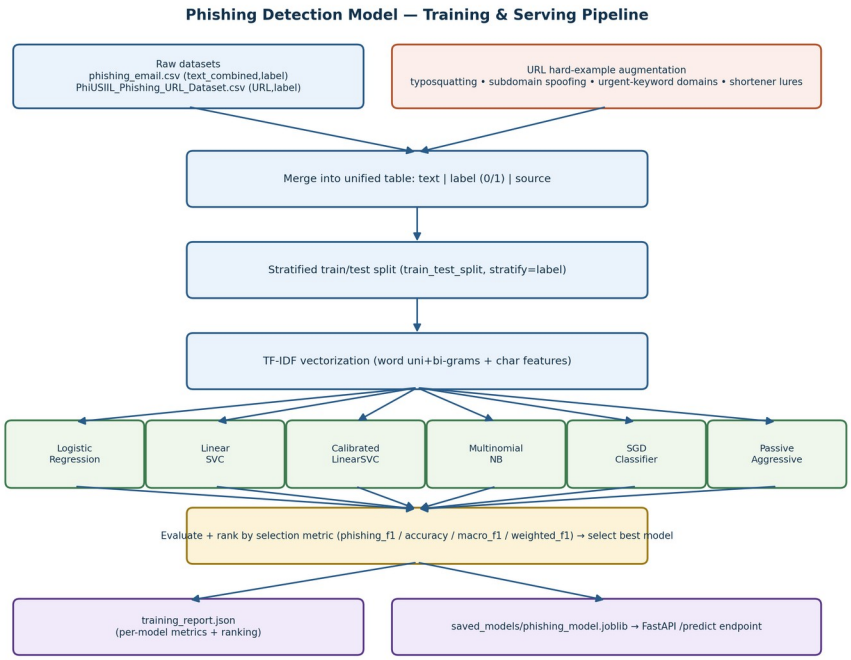


Figure 2.5: *Phishing detection model: training and serving pipeline (logbook sketch, weeks 3 to 7).*

The evaluation metrics were chosen for an imbalanced, security-sensitive problem. Accuracy alone is misleading, so the main selection metric was the **F1 score of the phishing class**:

Table 2.1: *Classification metrics used to evaluate and rank the phishing models (TP = true positive, FP = false positive, FN = false negative, TN = true negative).*

Metric	Formula	What it tells us
Accuracy	(TP + TN) / all predictions	Share of all predictions that are correct
Precision	TP / (TP + FP)	Of the messages flagged as phishing, how many really are
Recall	TP / (TP + FN)	Of all phishing messages, how many were

		caught
F1 score	2 x Precision x Recall / (Precision + Recall)	Balance of precision and recall; used to rank models

Hard examples through data augmentation

Reviewing early results showed that the model missed disguised URLs such as `p4ypal.com` or `paypal.verify-123.xyz`. The public datasets simply did not contain enough of them. I wrote generators for **typosquatting** variants (character substitutions such as a to 4, o to 0, l to 1, m to rn and w to vv), brand-in-subdomain spoofs and urgent-keyword domains (Figure 2.6), plus a curated list of real brand domains as hard negatives so the model would not start flagging genuine sites. The augmentation set is controlled by a `--url-augmentation-weight` flag.

phishing-detector/train.py - typosquatting hard examples

```python
LOOKALIKES = {"a": "4", "o": "0", "l": "1", "e": "3", "i": "1", "m": "rn", "w": "vv"}
BRANDS = ["paypal", "google", "microsoft", "apple", "amazon", "facebook", "firstbank", "gtbank"]
URGENT = ["verify", "secure", "update", "login", "account", "suspended", "confirm"]

def typosquat_variants(brand: str) -> list[str]:
    """paypal -> p4ypal, payp4l, paypal ... one substitution at a time."""
    out = []
    for i, ch in enumerate(brand):
        if ch in LOOKALIKES:
            out.append(brand[:i] + LOOKALIKES[ch] + brand[i + 1:])
    return out

def make_hard_examples(seed: int = 42) -> pd.DataFrame:
    rng = random.Random(seed)
    rows = []
    for brand in BRANDS:
        for fake in typosquat_variants(brand):
            rows.append(f"https://{fake}.com/{rng.choice(URGENT)}")
        rows.append(f"http://{brand}.{rng.choice(URGENT)}-{rng.randint(100, 999)}.xyz/login")  # brand in subdomain
    return pd.DataFrame({"text": rows, "label": 1, "source": "augmented"})
```

Figure 2.6: *Generating typosquatting and brand-spoofing URLs as hard training examples.*

Safe retraining: champion and challenger

Retraining on new data should never make production worse. In week 13 I replaced the practice of overwriting a single model file with timestamped model versions and a `promote_model.py` script (Figure 2.7). A newly trained challenger replaces the production model only if it scores higher on the selection metric. I tested it with a deliberately weaker model to confirm that it correctly refused to promote.

phishing-detector/promote_model.py - champion / challenger

```python
def promote(challenger_path: Path, metric: str = "phishing_f1") -> bool:
    """Replace the production model only if the newly trained challenger is better."""
    champion = json.loads(Path("saved_models/production_report.json").read_text())
    challenger = json.loads(challenger_path.with_suffix(".json").read_text())
    old, new = champion["metrics"][metric], challenger["metrics"][metric]
    if new <= old:
        print(f"kept production model: {metric} {new:.4f} <= {old:.4f}")
        return False
    shutil.copy(challenger_path, "saved_models/phishing_model.joblib")
    shutil.copy(challenger_path.with_suffix(".json"), "saved_models/production_report.json")
    print(f"promoted {challenger_path.name}: {metric} {old:.4f} -> {new:.4f}")
    return True
```

Figure 2.7: *Champion/challenger promotion: the production model is only replaced by a better one.*

Generative AI: Acadeva's content tools

From week 8 I joined the EdTech sub-team, which uses Google's Gemini models through carefully engineered prompts. All four tools follow the same pattern (Figure 2.8): a book or article goes in with a system prompt, and a structured JSON object comes out.

- **Acadeva Magic:** generates book metadata (title, description, course code, terms, subject, category, quiz type, handwritten and explicit-content flags) from an uploaded PDF. I improved the few-shot examples to reduce hallucinated course codes, fixed multi-course-code parsing, improved handwritten detection and tightened explicit-content handling.
- **Book Topic Ranker:** scores a book's relevance (0.00 to 1.00) against a fixed list of 140+ topics. I refined the topic list and the scoring-consistency instructions.
- **Content Categorizer:** assigns one category to an article or post using few-shot examples. I curated examples for local news and fixed mishandling of sponsored content.
- **Quithn:** generates a 25-question quiz with options, correct answer, feedback and difficulty from a chapter. I improved the difficulty and JSON-validation instructions and fixed a duplicate-answer-option bug.

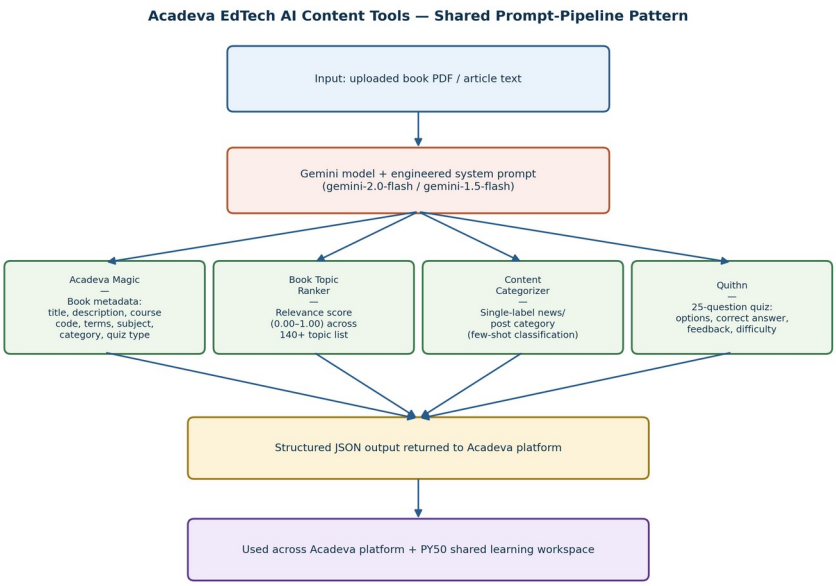


Figure 2.8: *Acadeva EdTech AI content tools: the shared prompt-pipeline pattern (logbook sketch, weeks 8 to 10).*

The most important lesson from this work is that a language model's output must be treated as untrusted until it is validated: it can invent values that look plausible. Asking for structured JSON, giving clear few-shot examples, setting a low temperature and validating the output against a schema all reduce this risk. I applied the same lesson, more strictly, in the AI voice scribe of my IT project (Chapter Three).

- **Practical activity:** TF-IDF feature extraction, six-model comparison, probability calibration, augmentation, champion/challenger retraining, prompt engineering for four LLM tools.
- **Tools/technologies used:** scikit-learn, joblib, Gemini API (gemini-2.0-flash / 1.5-flash), JSON schema output.
- **Skills acquired:** model selection and evaluation on imbalanced data, feature engineering for text, error analysis, safe deployment, prompt engineering.

2.7 THE DATA SCIENCE WORKFLOW

The most valuable thing I learned during the placement is not any single tool, but the **workflow** that connects them. A data science project is a cycle of well-defined steps, and skipping a step almost always causes problems later. Figure 2.9 shows the workflow as I learned and practised it.

It is closely related to the industry-standard CRISP-DM process (Cross-Industry Standard Process for Data Mining).

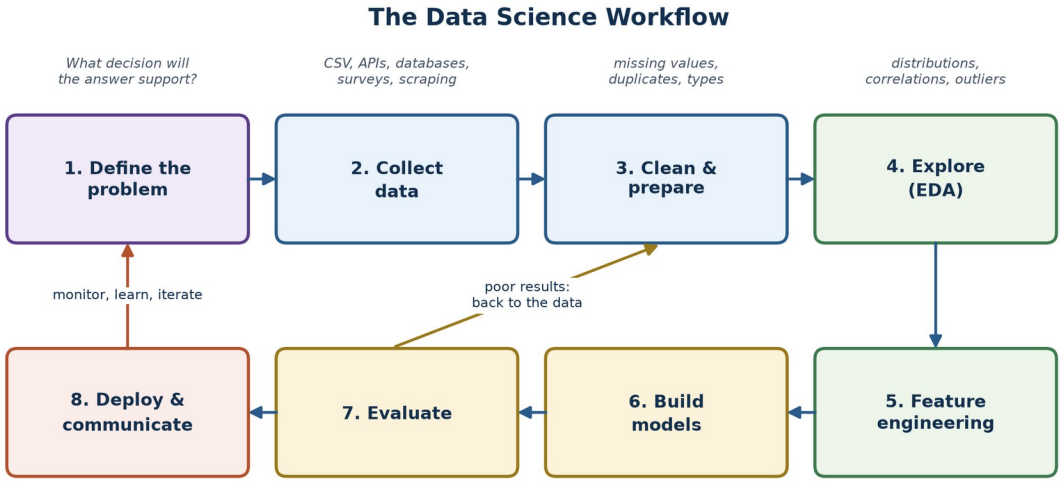


Figure 2.9: *The data science workflow: eight steps in a cycle, with feedback loops.*

1. **Define the problem:** state the question in terms of a decision. "Flag phishing messages before a student clicks them" is a problem; "use machine learning" is not. Decide how success will be measured (for example, phishing-class F1).
2. **Collect data:** find or gather data that can answer the question: files, databases, APIs, surveys or logs. Record where it came from and its licence.
3. **Clean and prepare:** handle missing values, remove duplicates, fix types and inconsistent labels, and join sources. This step usually takes the most time.
4. **Explore (EDA):** study distributions, class balance, correlations and outliers with summary statistics and charts. EDA often changes the plan.
5. **Feature engineering:** turn raw data into informative inputs, for example TF-IDF weights from text, family size from two columns, or rooms per household.
6. **Build models:** start with a simple baseline, then try stronger models. Always split data into training and test sets, and use cross-validation where possible.
7. **Evaluate:** measure performance on data the model has never seen, with metrics that fit the problem, and analyse the errors. Poor results send you back to the data, not just to a different algorithm.
8. **Deploy and communicate:** put the model where it is used (an API, a dashboard, a report), explain results to non-technical people, then monitor it and retrain safely as new data arrives.

Table 2.2: *Mapping the data science workflow to the phishing detection project.*

Workflow step	How it was applied in the phishing detection project
1. Define the problem	Acadeva needed an in-house model to flag phishing emails and URLs; success = phishing-class F1
2. Collect data	phishing_email.csv and the PhiUSIIL URL dataset
3. Clean and prepare	Label normalisation, merging, de-duplication, source column
4. Explore	Class balance per source; found weakness on disguised URLs
5. Feature engineering	TF-IDF word uni/bi-grams + character features; augmented hard examples
6. Build models	Logistic Regression baseline, then six ranked candidates
7. Evaluate	Accuracy, precision, recall, F1, per-source metrics, low-confidence review
8. Deploy and communicate	FastAPI /predict, README, training_report.json, champion/challenger retraining, team demos

2.8 BEGINNER DATA SCIENCE PROJECTS

To practise the complete workflow on well-known problems, I completed three classic beginner data science projects. They cover the three most common types of supervised learning problem: binary classification, regression and multi-class classification. All three were done in one Jupyter notebook (`report/notebooks/beginner_data_science_projects.ipynb`), and every number below comes from running it.

2.8.1 Project 1: Titanic Survival Prediction

Problem. Given a passenger's details, predict whether they survived the sinking of the Titanic in 1912. This is a binary classification problem. **Data.** 891 passengers with 15 columns, including class, sex, age, number of relatives on board, fare and port of embarkation. Only 38.4% survived.

Cleaning. Age was missing for 177 passengers. Instead of filling them all with one average, I filled each missing age with the median age of passengers of the same class and sex, which is more realistic. The two missing embarkation ports were filled with the most common port, and the `deck` column (77% missing) was dropped. **Feature engineering.** I created `family_size` (siblings/spouses + parents/children + 1) and `is_child` (age under 13), encoded sex as 0/1 and one-hot encoded the port.

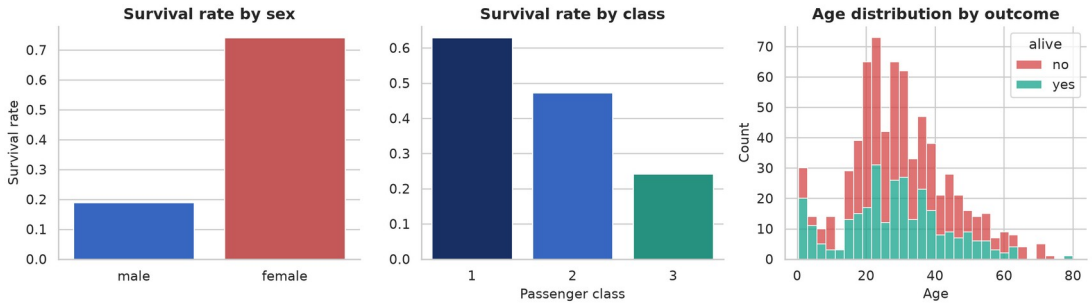


Figure 2.10: *Titanic EDA: survival by sex and by class, and age distribution by outcome.*

EDA (Figure 2.10) shows the famous "women and children first" pattern: 74.2% of women survived but only 18.9% of men, and survival fell from 63.0% in first class to 24.2% in third class. I trained three models on an 80/20 stratified split and also measured 5-fold cross-validated accuracy, which is more reliable on a small dataset (Table 2.3).

Table 2.3: *Titanic results.*

Model	Test accuracy	5-fold CV accuracy
Logistic Regression	81.6%	80.7%
Decision Tree (max depth 4)	79.9%	80.7%
Random Forest (300 trees)	80.4%	**82.2%**

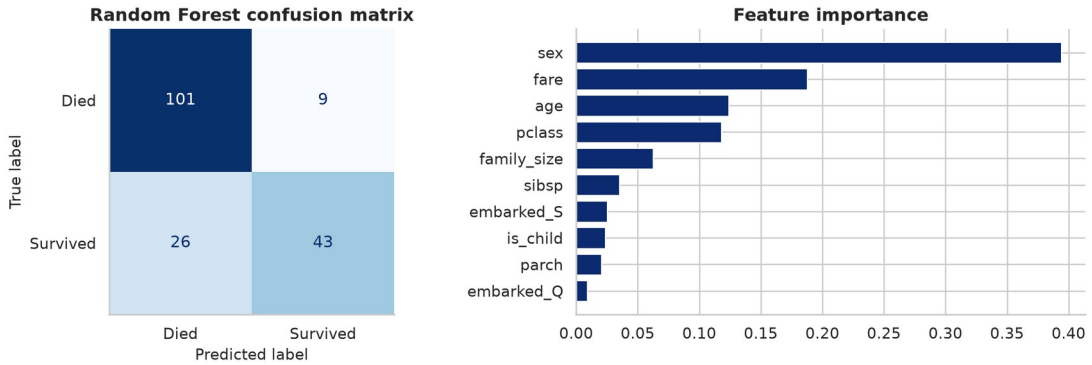


Figure 2.11: *Random Forest confusion matrix and feature importance for the Titanic model.*

Findings. The Random Forest had the best cross-validated accuracy (82.2%). Its feature importance (Figure 2.11) confirms the EDA: sex is by far the strongest predictor, followed by fare, age and class. The confusion matrix shows the model is better at recognising passengers who died (101 of 110) than those who survived (43 of 69), which is common when one class is larger.

2.8.2 Project 2: House Price Prediction

Problem. Predict the median house value of a district in California from its characteristics. This is a regression problem, because the target is a continuous number. **Data.** The California Housing dataset: 20,640 districts, with location, housing age, rooms, bedrooms, population, households, median income and proximity to the ocean.

Cleaning and features. 207 missing `total_bedrooms` values were filled with the median. Raw totals are not comparable between large and small districts, so I engineered ratios: rooms per household, bedrooms per room and population per household. The categorical `ocean_proximity` column was one-hot encoded.

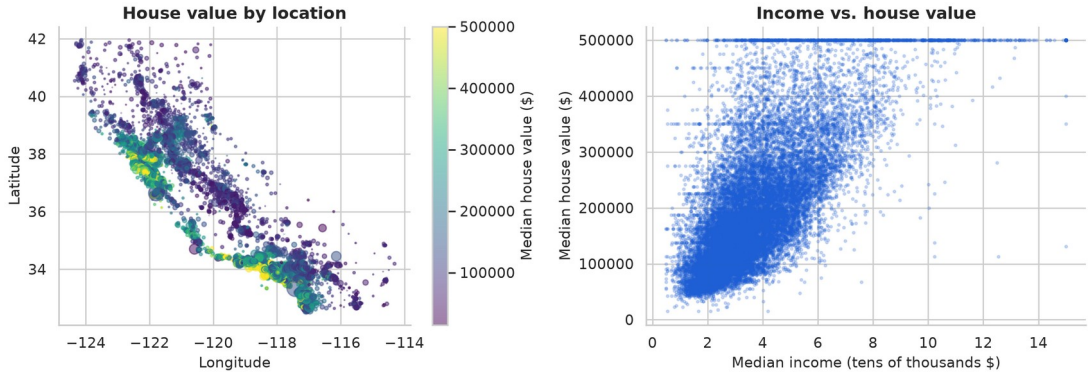


Figure 2.12: *California housing EDA: value by location (point size = population) and income vs. value.*

The map (Figure 2.12) shows that coastal districts around the Bay Area and Los Angeles are the most expensive, and median income is strongly related to house value (correlation 0.69). Notice the horizontal line at $500,000: values above that were capped when the data was collected, a data quality issue that limits any model's accuracy at the top end.

Table 2.4: *House price results on the 20% test set.*

Model	MAE	RMSE	R^2
Linear Regression	$50,889	$72,669	0.597
Random Forest (150 trees)	**$32,230**	**$50,131**	**0.808**

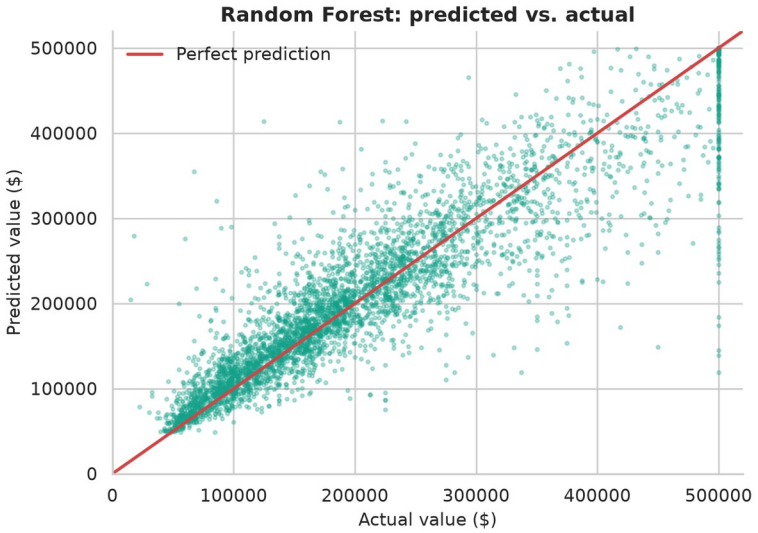


Figure 2.13: *Random Forest predicted vs. actual house values (points near the red line are accurate).*

Findings. The Random Forest explains about 81% of the variation in house values ($R^2 = 0.808$) and its average error is about $32,000, a 37% reduction compared with linear regression, because it can capture non-linear effects such as location. Figure 2.13 shows the predictions cluster around the perfect-prediction line, with the largest errors at the $500,000 cap.

2.8.3 Project 3: Iris Flower Classification

Problem. Identify which of three species (setosa, versicolor or virginica) an iris flower belongs to, from four measurements: sepal length and width, and petal length and width. This is a multi-class classification problem and the classic first project in machine learning. **Data.** 150 flowers, 50 of each species, with no missing values.

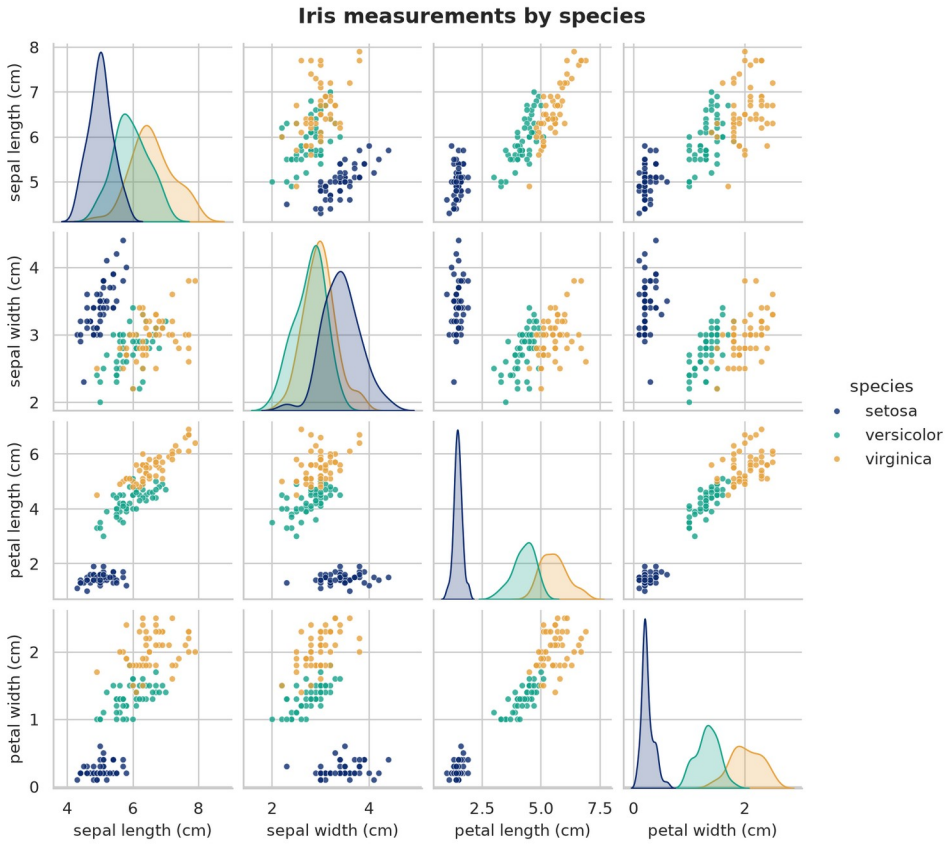


Figure 2.14: *Pair plot of the four iris measurements, coloured by species.*

The pair plot (Figure 2.14) shows that setosa is completely separated by petal size, while versicolor and virginica overlap slightly. I used **K-Nearest Neighbours (KNN)**, which classifies a flower by the majority species of its k closest neighbours. To choose k, I tested every value from 1 to 20 on a stratified 70/30 split (Figure 2.15).

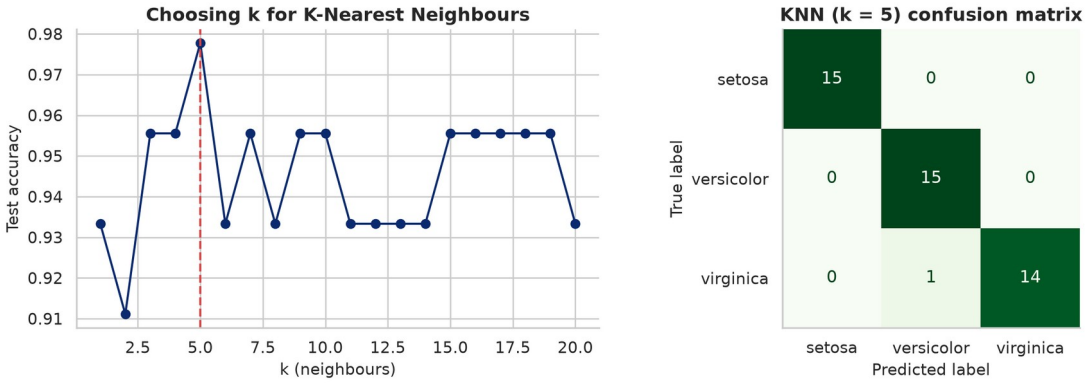


Figure 2.15: *Choosing k for K-Nearest Neighbours, and the confusion matrix of the final model.*

Findings. With k = 5 the model classified 44 of the 45 test flowers correctly (97.8% accuracy). The only error was one virginica predicted as versicolor, exactly where the EDA showed the two species overlap.

Table 2.5: *Summary of the three beginner data science projects.*

Project	Problem type	Best model	Result	Key lesson
Titanic survival	Binary classification	Random Forest	82.2% CV accuracy	Group-wise imputation and feature engineering
House prices	Regression	Random Forest	$R^2 = 0.808$, MAE $32,230	Ratios beat raw totals; watch for capped data
Iris species	Multi-class classification	KNN (k = 5)	97.8% accuracy	Tune hyper-parameters on held-out data

2.9 OTHER TECHNICAL AND PROFESSIONAL SKILLS

Beyond the technical work, the placement developed skills that are just as important in a professional environment:

- **Teamwork and code review:** every significant change, such as the shared-helper refactor and the explicit-content work, went through review with my supervisor before being merged.
- **Knowledge sharing:** in week 10 I walked PY50 trainees through the phishing-detection pipeline, and in week 24 I handed my work over to the next intern.
- **Technical documentation:** I wrote the project README, setup and contribution guides, a retraining guide, docs for the shared prompt helper and inline comments for complex code.
- **Prioritisation and scope discipline:** in week 19 I triaged a backlog of user-reported bugs by impact, and in week 21 I chose one meaningful improvement and finished it cleanly instead of starting several.
- **Integration testing:** in week 22 I ran the phishing API and the EdTech tools together end to end, which revealed a rate-limit conflict that unit tests could not catch.
- **Communication and presentation:** I presented the analytics view to the content team and delivered the end-of-placement demo to the Acadeva team.

2.10 CONCLUSION

This chapter has presented the practical skills I acquired at Acadeva, from collecting and cleaning data to building, evaluating, serving and maintaining machine learning models and

generative-AI tools. The data science workflow tied these activities together, and the three beginner projects demonstrated it end to end. The next chapter shows how I applied the same skills, in software engineering, data analysis and safe use of AI, to build my IT project: AraCare Health OS.

CHAPTER THREE

IT PROJECT ON ARACARE HEALTH OS

INTRODUCTION

This chapter presents the IT project I developed by applying the skills acquired during SIWES: **AraCare Health OS**, a hospital operating system for Nigerian health facilities. The project combines the software engineering practices I used at Acadeva (APIs, validation, automated tests, documentation), the data science workflow (data modelling, analysis and visualisation), and the safe use of AI (grounded, human-reviewed language-model output). The chapter covers the background and problem, the aim and objectives, the methodology and tools, the system architecture and design, a walkthrough of the working system from login to every major screen, including the waiting-room TV triage display, an analysis of the system's data in a Python notebook, testing, and finally the challenges, limitations and future work.

3.1 PROJECT OVERVIEW

AraCare Health OS is a modular hospital information system designed to run on **one server inside each hospital**, so that the hospital keeps working even when the internet is down. Staff use role-based web screens over the hospital's local network: reception registers and checks in patients, nurses take vital signs at triage, doctors write consultation notes and orders, and administrators manage staff and settings. A public TV screen in the waiting room shows which ticket numbers are being served and who should be ready next.

The first release, documented in this report, delivers four modules end to end: **patient registration and check-in**, the **queue engine**, the **electronic health record (EHR)** for triage and consultation, and an **AI voice scribe** that turns dictation into a draft note entirely on the hospital's own computer. Billing, pharmacy and inventory, laboratory, capacity and finance dashboards, internal communications and IoT patient monitoring are fully designed in the project's documentation (`docs/`), and the events they will need are already being published, but they are not yet built.

Table 3.1: *AraCare Health OS at a glance.*

Item	Details

Project title	AraCare Health OS: an offline-first hospital operating system
Type	Full-stack web application (backend API + web client) with data analysis
Backend repository	github.com/Aniebiet1/ARACARE-HEALTH-OS-BACKEND (API, design docs, screenshots)
Frontend repository	github.com/Aniebiet1/ARACARE-HEALTH-OS-FRONTEND (React web client)
Users	Receptionists, nurses, doctors, administrators, patients (receipt, SMS, TV display)
Release covered	Registration and check-in, queue engine, EHR (triage and SOAP notes), AI voice scribe
Size	About 4,000 lines of Python in the backend, 11 detailed design documents, 47 automated tests

3.2 BACKGROUND OF THE STUDY

Outpatient departments (OPD) in Nigerian public and private hospitals are under constant pressure. The design document for the queue engine describes a typical morning: around 200 patients arrive between 07:30 and 10:00, but three doctors who each see a patient every 8 to 12 minutes can see only about 60 patients in a session. The remaining patients wait for three to five hours. Many leave the premises to eat or run errands and lose their place, because the "queue" is a pile of paper folders on a desk. There is no reliable way to tell a patient how long they will wait, and no guarantee that a very sick patient is seen before a routine check-up.

Clinical documentation has similar problems. Records are kept on paper cards and folders that are hard to find, easy to lose and often illegible. Paper notes do not produce standard diagnosis codes (ICD-10) that insurance (HMO) claims need, and they cannot warn a doctor that a patient is allergic to the drug being prescribed. Where electronic systems exist, they often depend on a constant internet connection, which many facilities do not have, and typing structured notes takes time that busy clinicians do not have. Voice dictation could help, but cloud speech services are expensive, send patient audio outside the hospital, and perform poorly on Nigerian accents, Pidgin English and noisy wards.

These observations, together with what I learned at Acadeva about building reliable APIs and using AI safely, motivated the design of AraCare.

3.3 PROBLEM STATEMENT

Nigerian hospital outpatient departments lack an affordable, offline-capable system that can organise patient flow and capture structured clinical records safely. Specifically:

9. Queues are managed manually, so waiting is long, unpredictable and unfair, and urgent patients are not reliably prioritised.
10. Patients who step away lose their place, and there is no way to inform them when their turn is near.
11. Patient records are on paper, so they are lost, duplicated (the same patient registered several times with different spellings) and not searchable.
12. Clinical safety checks, such as abnormal vital signs and drug allergies, depend entirely on human memory.
13. Structured documentation is slow to type, and cloud-based voice tools are costly, need internet access and raise privacy concerns.

3.4 AIM AND OBJECTIVES

Aim: to design and implement an offline-first hospital operating system that organises outpatient flow fairly and captures structured, safe clinical records, with an optional local AI assistant for documentation.

Objectives:

14. To build a patient registration and check-in module with unique patient numbers, duplicate detection, fuzzy name search and QR-coded receipts.
15. To build a queue engine that prioritises patients by clinical need, prevents starvation of lower-priority patients, and shows live queue status on a waiting-room TV display.
16. To build an EHR for triage vital signs and SOAP consultation notes with ICD-10 diagnoses, laboratory and drug orders, automatic vital-sign flags and an allergy hard-block.
17. To build an AI voice scribe that runs locally, never writes to the record without a clinician's signature, and removes values that were not actually spoken.
18. To secure the system with role-based access, audit logging and safe production settings, and verify it with automated tests.

19. To analyse the data produced by the system in order to evaluate whether the queue rules work and to describe the patient population.

3.5 SCOPE AND SIGNIFICANCE

The project covers the outpatient journey from arrival at reception to the end of the doctor's consultation, for a single facility running its own server. Laboratory and pharmacy steps are recorded as "deferred" journey steps so that visits complete cleanly now and the data is ready when those modules are added. Real SMS delivery is out of scope: messages are queued in an `outbound_messages` table, ready for an SMS gateway.

The project is significant because it shows that a modern, safe hospital system does not require expensive cloud subscriptions or a constant internet connection. It reduces waiting-room crowding by letting patients wait elsewhere and return when called, protects patients with automatic safety checks, produces coded data for insurance claims and public-health reporting, and keeps patient data inside the hospital.

3.6 METHODOLOGY AND TOOLS

The project followed an **incremental, design-first** approach. Before writing code, I wrote a detailed design document for each module (eleven documents in the `docs/` folder, covering registration, queues, EHR, the voice scribe, billing, inventory, capacity and finance dashboards, internal communications and IoT monitoring). Each document describes the real-world problem, the state machines, the data model and the edge cases. I then built the system as a **vertical slice**: a thin but complete path through registration, queues, EHR and scribe, rather than many half-finished modules. Every rule that protects patients was written as an automated test, and the user interface was checked by walking through a full visit and capturing the screenshots shown in Section 3.9.

Table 3.2: *Technology stack of AraCare Health OS.*

Layer	Technology	Why it was chosen
Backend API	Python 3.11, FastAPI, Uvicorn	Fast, automatic validation and API docs (as used at Acadeva)
Data validation	Pydantic v2	Typed request/response models; validates AI output
Database access	SQLAlchemy 2.0, Alembic	One codebase for SQLite and PostgreSQL; versioned migrations

Database	SQLite (development), PostgreSQL 16 (production)	Zero-setup locally; robust in production
Authentication	JWT (PyJWT), salted password hashing	Stateless tokens; lockout after failed logins
Real-time updates	WebSockets (FastAPI)	Queue screens and TV refresh instantly
Frontend	React 18, Vite, jsQR, qrcode.react	Fast single-page app; QR receipts and camera scanning
Speech to text	faster-whisper (OpenAI Whisper weights)	Runs locally on CPU; MIT licence; no cost
Note drafting	Ollama with qwen2.5:7b, rule-based fallback	Local LLM; keeps working if the model is absent
Testing	pytest, FastAPI TestClient	47 automated tests of rules and full visit flows
Deployment	Docker, Docker Compose	One command to run the API and PostgreSQL on a facility server
Data analysis	Jupyter, pandas, NumPy, Matplotlib, Seaborn	Notebook analysis of system records (Section 3.10)

3.7 SYSTEM ARCHITECTURE

AraCare is a **modular monolith**: one deployable API process per facility, internally divided into modules that each own their own database tables (Figure 3.1). This is simpler to deploy and maintain on a single hospital server than many separate microservices, while keeping the code organised so that modules could be separated later. Role-based web clients talk to the API over the hospital's local network using REST, and a WebSocket channel pushes queue changes to every screen instantly.

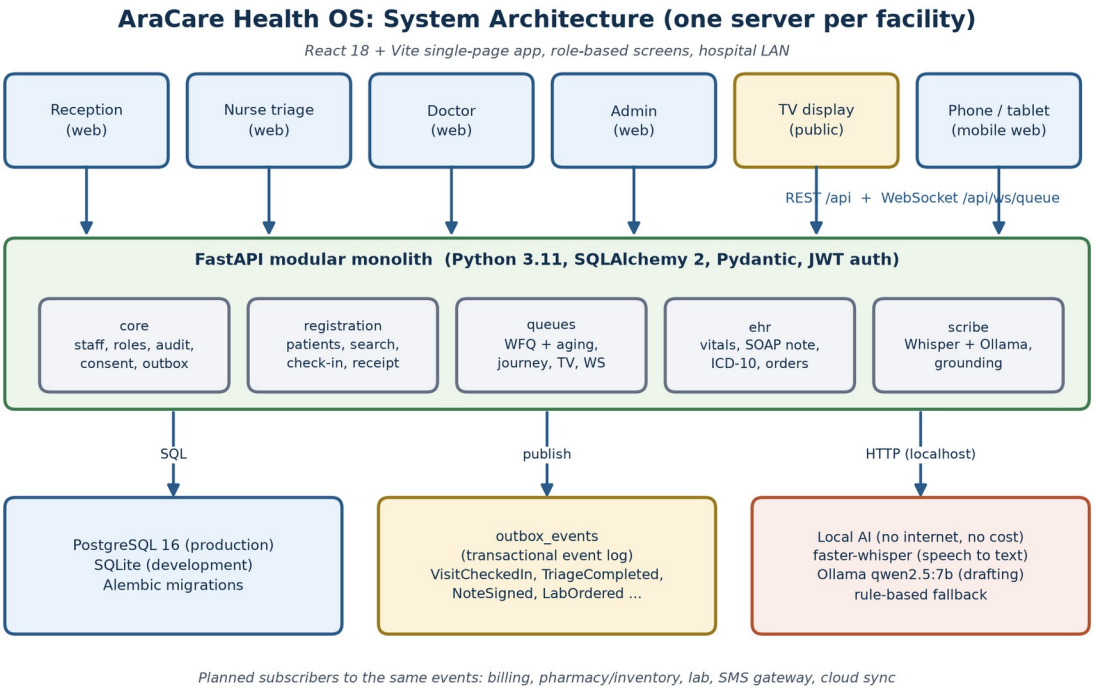


Figure 3.1: *System architecture of AraCare Health OS.*

Modules never call each other's internals. Instead they communicate through **domain events** such as `VisitCheckedIn`, `TriageCompleted` and `NoteSigned`. Each event is written to an `outbox_events` table in the same database transaction as the change that caused it, so an event can never be lost or published for a change that was rolled back (Table 3.3). The same table will later feed SMS dispatch, cloud backup and the billing and pharmacy modules.

Table 3.3: *Domain events: the only coupling between modules.*

Event	Published by	Reacted to by (now)	Planned subscribers
PatientRegistered	registration	messaging (card SMS)	
VisitCheckedIn	registration	queues (journey + triage ticket)	billing: draft invoice
TriageCompleted	ehr	queues (doctor ticket, set priority)	
CriticalVitals	ehr	real-time alert	on-call notification
LabOrdered / Prescribed	ehr (on sign)	queues (deferred LAB/PHARM step)	lab, pharmacy, inventory
NoteSigned	ehr	queues (close doctor step)	billing charge, HMO claim (ICD-10)
JourneyClinicalCompleted	queues	log	billing: cashier ticket

All data hangs off one **spine** (Figure 3.2): a Patient has many Visits; each Visit has one Journey made of ordered JourneySteps (TRIAGE, OPD-GEN, LAB, ...), and each live step has a QueueTicket. Clinical records (vitals, the note, diagnoses and orders) attach to the Visit, so every future charge, stock movement or insurance claim will attach to the same visit too.

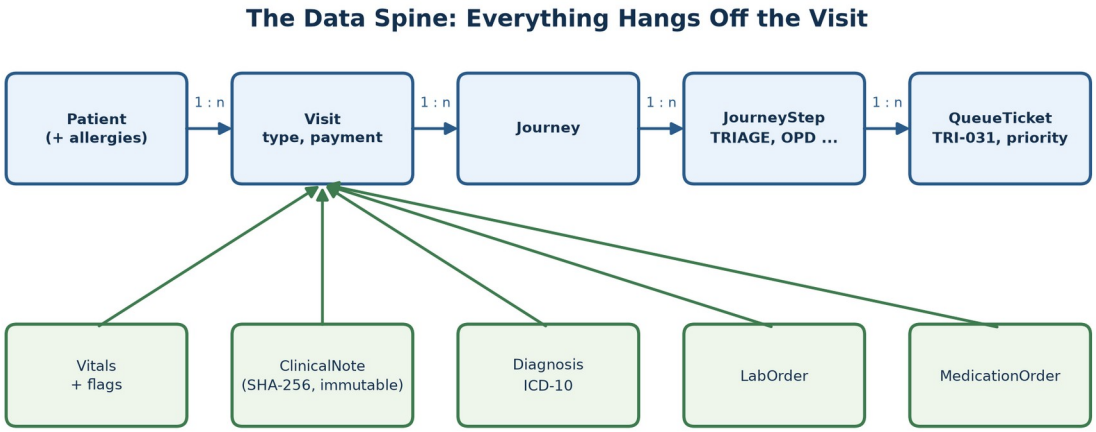


Figure 3.2: *The data spine: Patient, Visit, Journey, JourneyStep and QueueTicket, with clinical records.*

Figure 3.3 shows how a patient moves through the system and which event moves them to the next step.

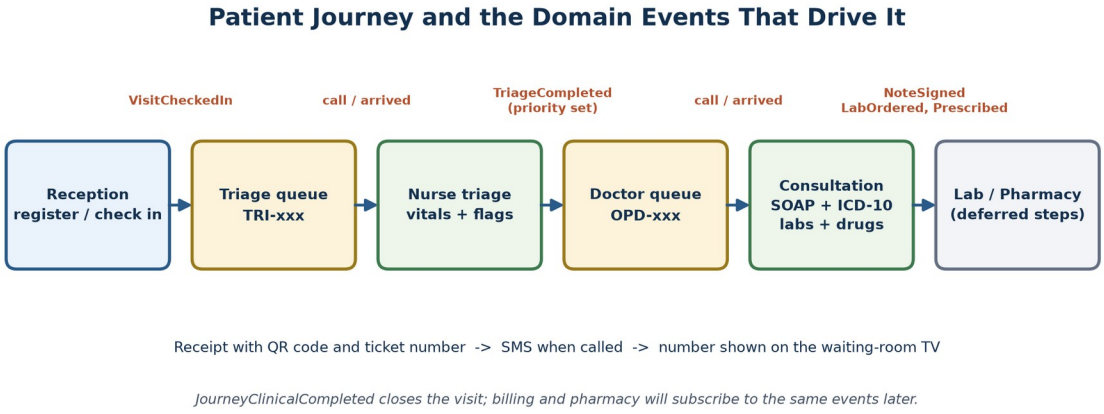


Figure 3.3: *The patient journey and the domain events that drive it.*

3.8 DESIGN AND IMPLEMENTATION OF THE CORE MODULES

3.8.1 Patient Registration and Check-in

Each patient receives a facility-scoped number such as `ARC-LAG-2026-000301` (system, state code, year, sequence), plus a global UUID so that records can be shared across facilities in future. Numbers are issued from a row-locked counter, so two receptionists registering patients at the same moment can never receive the same number (this is covered by a concurrency test). Before saving a new patient, the system checks for likely duplicates by phone number and similar name.

Searching is designed for real Nigerian data, where the same name is spelled in different ways (Okonkwo/Okonkow, Babatunde/Babtunde) and phone numbers are written as 0803..., +234803... or 234803.... The search normalises phone numbers and scores each name token with a fuzzy string similarity (`difflib.SequenceMatcher`), so near-misses still match. A scanned QR code or typed patient number goes straight to the exact record.

Checking in creates a Visit with a reason (new complaint, follow-up, antenatal, emergency) and a payment track (out of pocket, HMO, government, staff, waiver; waivers need administrator approval). The `VisitCheckedIn` event starts the journey and issues a triage ticket, and the receptionist prints a receipt with the ticket number and a QR code the patient can show on the next visit.

3.8.2 The Queue Engine

Every department has its own daily queue, and every patient in it is a **ticket** with exactly one state at a time (Figure 3.4). A ticket is WAITING until a staff member clicks "Call next patient"; it is then CALLED (an SMS is queued and the TV shows the number); when the patient appears it becomes IN_SERVICE, then COMPLETED. If the patient does not appear in time, a background sweeper marks the ticket NO_SHOW; the patient can rejoin once, and a second no-show cancels the ticket.

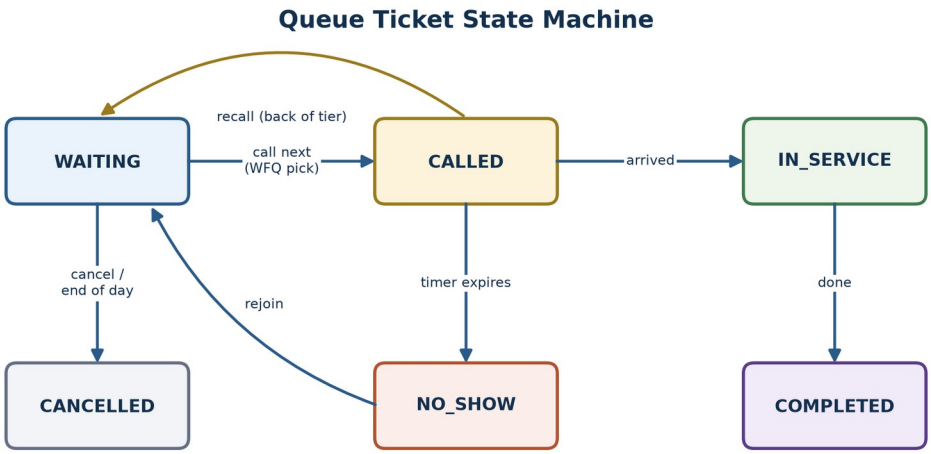


Figure 3.4: *Queue ticket state machine.*

The starting priority comes from the visit, not from guesswork (Figure 3.5): emergency visits are EMERGENCY; follow-ups are FOLLOW_UP; antenatal visits, children under five and adults aged 65 or over are HIGH; everyone else is NORMAL. After triage, the nurse's assessment and the vital-sign rules can raise the priority for the doctor queue, and critical vital signs force EMERGENCY.

backend/app/modules/queues/service.py - initial priority

```python
def initial_priority(patient: Patient, visit: Visit) -> str:
    if visit.visit_type == "emergency":
        return "EMERGENCY"
    if visit.visit_type == "follow_up":
        return "FOLLOW_UP"
    age = age_years(patient.date_of_birth)
    if visit.visit_type == "anc" or age < 5 or age >= 65:
        return "HIGH"
    return "NORMAL"
```

Figure 3.5: *Initial priority rules (queues/service.py).*

A simple priority queue would let a steady stream of HIGH patients keep a NORMAL patient waiting all day. AraCare prevents this with two techniques (Figure 3.6):

- **Aging:** a NORMAL ticket that has waited more than 45 minutes is treated as HIGH, and a FOLLOW_UP ticket that has waited more than 90 minutes is treated as NORMAL. Aging is computed on read and never stored back, so it cannot snowball.

- **Weighted fair queuing (WFQ):** after 3 HIGH patients in a row, the next call goes to a waiting NORMAL patient; after 5 NORMAL patients in a row, a FOLLOW_UP patient gets a turn. EMERGENCY always goes first.
- **Concurrency safety:** the department's queue state row is locked before choosing, so two consulting rooms clicking at the same moment can never call the same patient.

backend/app/modules/queues/service.py - aging + weighted fair queuing

```python
def effective_tier(t: Ticket, now: datetime | None = None) -> str:
    """Aging is computed from the base tier, never stored back: a FOLLOW_UP ticket ages one step to NORMAL,
    it does not then age again to HIGH, and reading a queue never writes to it."""
    now = now or utcnow()
    waited = (now - t.entered_waiting_at).total_seconds() / 60
    if t.priority_tier == "NORMAL" and waited > settings.aging_p2_minutes:
        return "HIGH"
    if t.priority_tier == "FOLLOW_UP" and waited > settings.aging_p3_minutes:
        return "NORMAL"
    return t.priority_tier


def waiting_tickets(db: Session, facility_id: str, dept: str) -> list[Ticket]:
    now = utcnow()
    rows = db.scalars(select(Ticket).where(Ticket.facility_id == facility_id, Ticket.department == dept,
                                           Ticket.day == local_today(), Ticket.status == "waiting")).all()
    return sorted(rows, key=lambda t: (TIER_RANK[effective_tier(t, now)], t.entered_waiting_at))


def pick_next(db: Session, facility_id: str, dept: str) -> Ticket | None:
    """Emergency first -> aging -> weighted fair queuing so HIGH can't starve NORMAL (or NORMAL starve FOLLOW_UP)."""
    st = _state(db, facility_id, dept, local_today())   # lock first: two counters can't pick the same patient
    rows = waiting_tickets(db, facility_id, dept)
    if not rows:
        return None
    now = utcnow()
    by = {tier: [t for t in rows if effective_tier(t, now) == tier] for tier in TIER_RANK}
    if by["EMERGENCY"]:
        return by["EMERGENCY"][0]
    if by["HIGH"] and not (st.p1_streak >= settings.p1_burst_limit and by["NORMAL"]):
        st.p1_streak += 1
        return by["HIGH"][0]
    if by["NORMAL"] and not (st.p2_streak >= settings.p2_burst_limit and by["FOLLOW_UP"]):
        st.p1_streak = 0
        st.p2_streak += 1
        return by["NORMAL"][0]
    if by["FOLLOW_UP"]:
        st.p2_streak = 0
        return by["FOLLOW_UP"][0]
    return by["HIGH"][0] if by["HIGH"] else None
```

Figure 3.6: *Aging, ordering and weighted fair queuing in pick_next() (queues/service.py).*

For example, if the triage queue holds H1, H2, H3, H4 (HIGH) and N1 (NORMAL), the calls go H1, H2, H3, then N1 because the HIGH burst limit of 3 was reached, then H4. Each waiting ticket's estimated wait is its position in the queue multiplied by the median service time of the last ten patients, divided by the number of counters active in the last hour; this estimate is shown on the TV display.

3.8.3 Electronic Health Record and Clinical Safety

The EHR records triage vitals (blood pressure, pulse, respiratory rate, temperature, SpO2, weight, height, BMI and pain score) and the doctor's SOAP note (Subjective, Objective, Assessment, Plan) with ICD-10 diagnoses, laboratory orders and prescriptions. Safety rules live

in the server code, not only in the user interface, so they cannot be bypassed by a different client. The clinical logic is kept in a separate file of pure functions with no database access, which makes it easy to test and to defend.

Vital-sign flags. Every saved set of vitals is checked against thresholds (Figure 3.7): for example a temperature above 37.5 °C is a fever, and 39.5 °C or more is critical; SpO2 below 94% is abnormal and below 90% is critical. Any critical flag publishes a `CriticalVitals` alert and forces EMERGENCY priority in the doctor queue.

backend/app/modules/ehr/clinical_rules.py - vital-sign flags

```python
def flag_vitals(v: dict) -> tuple[list[dict], bool]:
    """Returns (flags, is_critical). Thresholds follow the EHR spec (fever>37.5, SpO2<94 ...)."""
    flags, critical = [], False

    def add(field, label, level):
        nonlocal critical
        flags.append({"field": field, "label": label, "level": level})
        critical = critical or level == "critical"

    t, sp, hr, rr = v.get("temperature"), v.get("spo2"), v.get("pulse_rate"), v.get("respiratory_rate")
    sys_, dia = v.get("bp_systolic"), v.get("bp_diastolic")
    if t is not None:
        if t >= 39.5 or t < 35.0:
            add("temperature", f"Temperature {t}°C", "critical")
        elif t > 37.5:
            add("temperature", f"Fever {t}°C", "abnormal")
    if sp is not None:
        if sp < 90:
            add("spo2", f"SpO2 {sp}%", "critical")
        elif sp < 94:
            add("spo2", f"Low SpO2 {sp}%", "abnormal")
    if sys_ is not None:
        if sys_ >= 180 or (dia or 0) >= 120 or sys_ < 80:
            add("bp", f"BP {sys_}/{dia}", "critical")
        elif sys_ >= 140 or (dia or 0) >= 90 or sys_ < 90:
            add("bp", f"BP {sys_}/{dia}", "abnormal")
    if hr is not None:
        if hr > 130 or hr < 40:
            add("pulse_rate", f"Pulse {hr}", "critical")
        elif hr > 100 or hr < 50:
            add("pulse_rate", f"Pulse {hr}", "abnormal")
    if rr is not None:
        if rr > 30 or rr < 8:
```

Figure 3.7: *Vital-sign flagging rules (ehr/clinical_rules.py).*

Allergy hard-block. When a doctor prescribes a drug, the server checks it against the patient's recorded allergies by **drug class**, not just by name (Figure 3.8). A patient recorded as allergic to "penicillin" is therefore blocked from receiving amoxicillin or Augmentin, and "sulfa drugs" blocks Septrin. The prescription is refused unless the doctor documents an override reason, which is stored and audited.

backend/app/modules/ehr/clinical_rules.py - allergy hard block

```python
def drug_class_of(name: str) -> str | None:
    n = name.lower()
    for cls, members in DRUG_CLASSES.items():
        if any(m in n for m in members + CLASS_ALIASES.get(cls, [])):
            return cls
    return None


def allergy_conflicts(drug_name: str, allergies) -> list:
    """Hard-block check. Matches by drug class (amoxicillin vs 'penicillin' allergy) or by name."""
    cls = drug_class_of(drug_name)
    hits = []
    for a in allergies:
        # re-derive the class from the name: records saved before an alias existed still match
        a_cls = (a.allergen_name and drug_class_of(a.allergen_name)) or (a.drug_class or "").lower()
        if (cls and a_cls == cls) or \
           (a.allergen_name and a.allergen_name.lower() in drug_name.lower()):
            hits.append(a)
    return hits
```

Figure 3.8: *Drug-class allergy matching (ehr/clinical_rules.py).*

Signed notes are immutable. A note cannot be signed without an assessment and at least one ICD-10 diagnosis. Once signed, its content is fingerprinted with a SHA-256 hash and can no longer be edited; corrections are added as dated addenda, which is what medico-legal practice requires.

3.8.4 The AI Voice Scribe

The voice scribe lets a nurse or doctor speak instead of type. Its design applies the lesson I learned at Acadeva, that language-model output must be treated as untrusted, much more strictly, because here a hallucinated number could harm a patient. Figure 3.9 shows the pipeline.

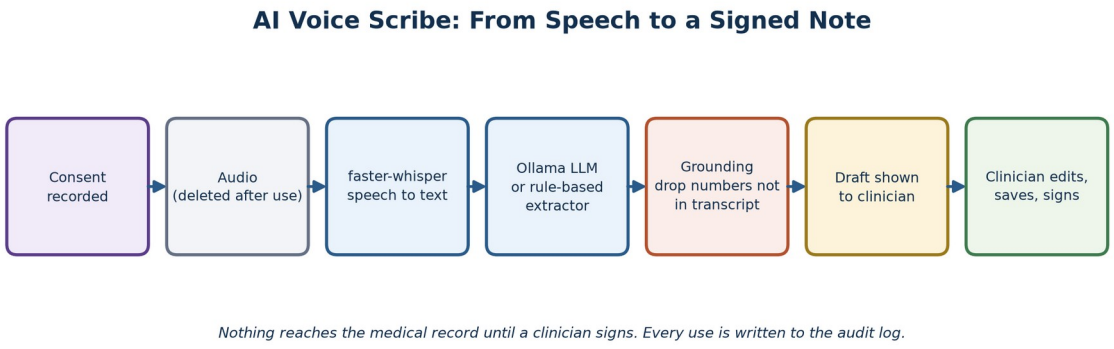


Figure 3.9: *AI voice scribe pipeline: from speech to a signed note.*

- **Local only:** speech is transcribed by faster-whisper and the draft is written by a local model served by Ollama, so no audio or text leaves the hospital and there is no per-use cost.

- **Consent first:** the patient's consent to recording is stored (append-only) before any audio is processed, and the audio file is deleted right after transcription.
- **Grounding:** any vital-sign number in the AI draft that does not literally appear in the transcript is removed and reported as dropped (Figure 3.10).
- **Draft, never record:** the scribe only fills the form. Suggested diagnoses, tests and drugs appear as chips that the clinician must accept, and the allergy check still runs on every accepted drug. Nothing is saved until the clinician signs.
- **Graceful degradation:** if Whisper is not installed, dictation can be typed or pasted; if Ollama is not running, a rule-based extractor takes over. The manual forms always work.

backend/app/modules/scribe/structure.py - grounding the AI draft

```python
_NUM = re.compile(r"\d+(?:\.\d+)?")


def _numbers(text: str) -> set[float]:
    return {float(x) for x in _NUM.findall(text)}


def ground_vitals(vitals: NurseVitals, transcript: str) -> tuple[NurseVitals, list[str]]:
    """Drop any numeric value that does not literally appear in the transcript."""
    present, dropped, clean = _numbers(transcript), [], vitals.model_dump()
    for field, val in vitals.model_dump().items():
        if val is not None and float(val) not in present:
            clean[field] = None
            dropped.append(f"{field}={val} (not found in transcript)")
    return NurseVitals(**clean), dropped
```

Figure 3.10: *Grounding: removing numbers the clinician never said (scribe/structure.py).*

3.8.5 Events, Security and Production Safety

The event bus (`app/events.py`) writes each event to the outbox, runs the subscribed handlers in the same transaction, and only broadcasts WebSocket notifications **after** the transaction commits, so screens never show a change that was rolled back. WebSocket messages carry only the event name, never patient data.

Security measures include role-based permissions (a nurse cannot write a doctor's note; a doctor cannot register patients), JWT authentication, account lockout for 15 minutes after five wrong passwords, forced password change for new and reset accounts, sign-out everywhere when a password or role changes, an audit log entry for every view of a chart, and a public TV display that shows only ticket numbers and initials. In production mode the server refuses to start with a weak secret key or a wildcard CORS origin, and no demo accounts are created.

3.9 SYSTEM WALKTHROUGH: USER INTERFACE

This section walks through the working system in the order a visit happens, from signing in to a signed consultation note, followed by the administrator's screens. All screenshots were taken from the running application with the demo facility "General Hospital Ikeja"; the patients are synthetic. Where two screens are shown side by side, the caption describes the left one first.

3.9.1 Sign In and Account Security

Every staff member signs in with a username and password (Figure 3.11). A wrong password shows a neutral message that does not reveal whether the username exists, and repeated failures lock the account for 15 minutes. Passwords can be changed from any screen; new passwords must be at least eight characters and mix letters with numbers or symbols, and changing one signs the user out on all other devices.

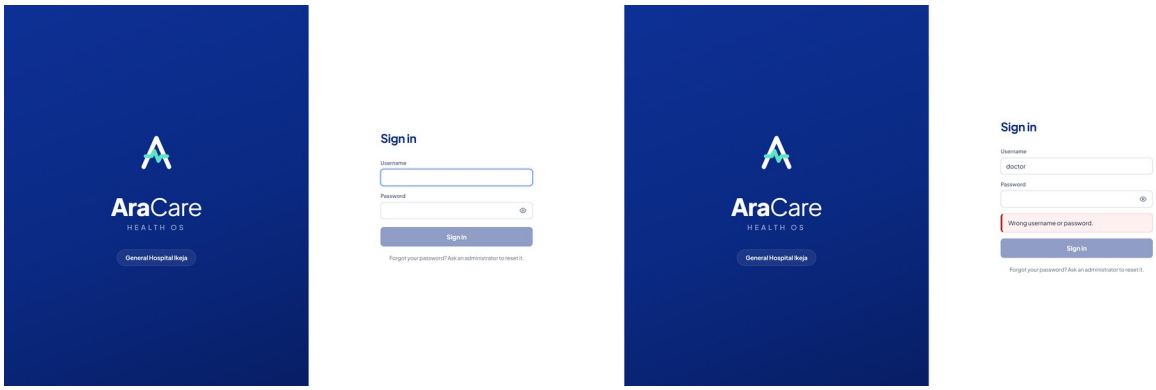


Figure 3.11: *Sign-in page (left) and the message shown after a wrong password (right).*

3.9.2 Waiting-Room TV Display (Triage and Doctor Queues)

The TV display is a public page meant for a large screen in each waiting area. Figure 3.12 shows the **triage** display: the ticket being served and its triage bay on the left, the next patients with their estimated wait on the right, and the day's statistics (waiting, served today, average wait) at the bottom. To protect privacy it shows only initials and surnames, never full names or clinical information. Figure 3.13 shows the same display for the General Outpatient (doctor) clinic, with two patients being served at once.

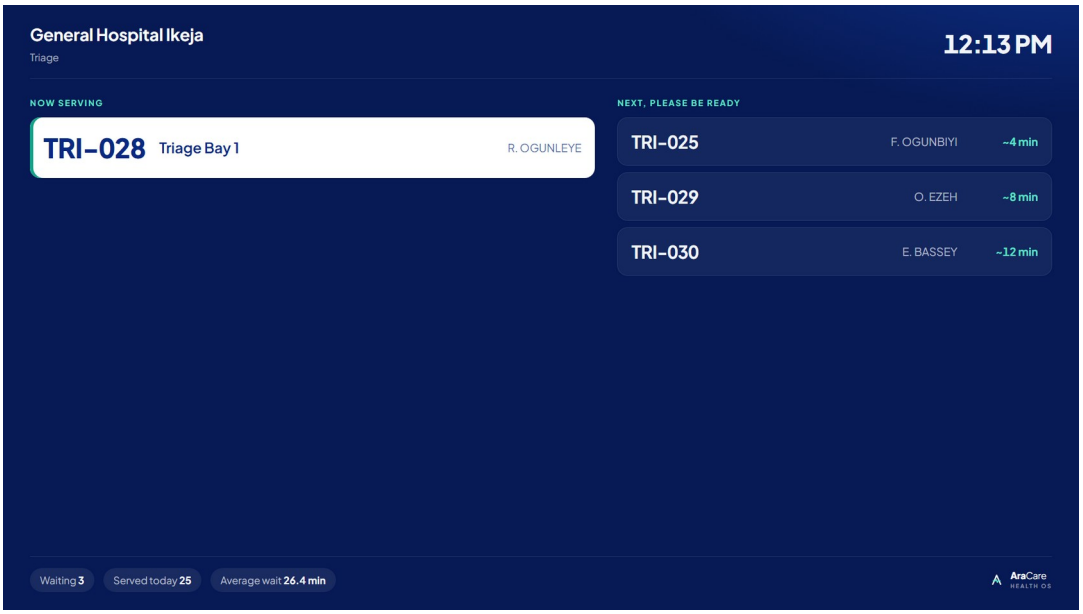


Figure 3.12: *Waiting-room TV display for the triage queue.*

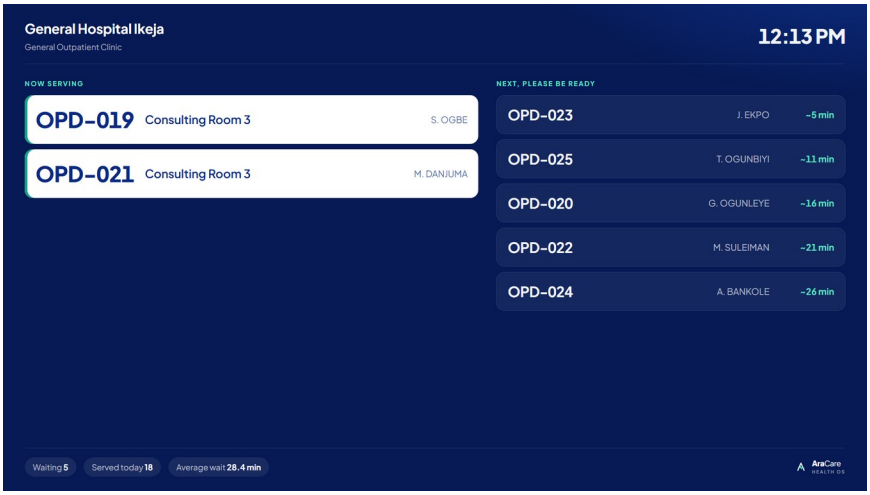


Figure 3.13: *Waiting-room TV display for the doctor (OPD) queue.*

The display (a React component, `frontend/src/pages/TV.jsx`) refreshes instantly through the WebSocket whenever any ticket changes, and every 10 seconds as a fallback.

3.9.3 Reception: Finding, Registering and Checking In Patients

The receptionist's home screen offers two ways to find a returning patient: scanning the QR code on their receipt, or searching by phone number, patient number or name. A fuzzy search for "Okafor" returns Okafor, Okoro and Okon records ranked by similarity. Selecting a patient opens their profile with the check-in form, while a new patient is registered with a short form in which only surname, first name, phone, date of birth and sex are required (Figure 3.14).

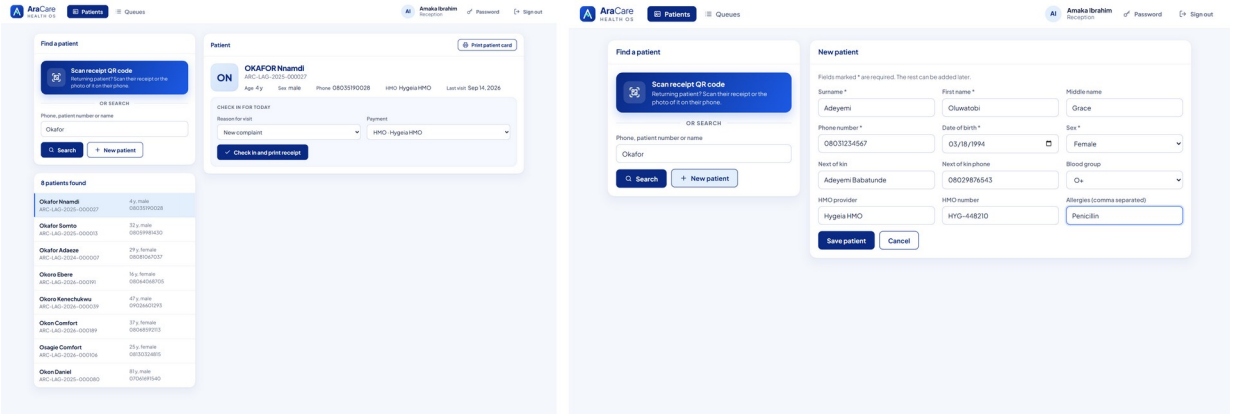


Figure 3.14: *Patient profile with the check-in form (left) and the new patient registration form (right).*

On saving, the patient number is generated and sent by SMS, known allergies are highlighted in amber, and the receptionist can ask for consent to take a photo.

Every patient can be given a printable **patient card** with a QR code, and each check-in produces a **visit receipt** showing the ticket number (here TRI-031), where to go next, payment details and allergies (Figure 3.15). On the next visit the receptionist scans the code with a webcam, a handheld scanner, or a photo of the receipt on the patient's phone, and reception keeps a live overview of both queues with each patient's priority and waiting time (Figure 3.16).

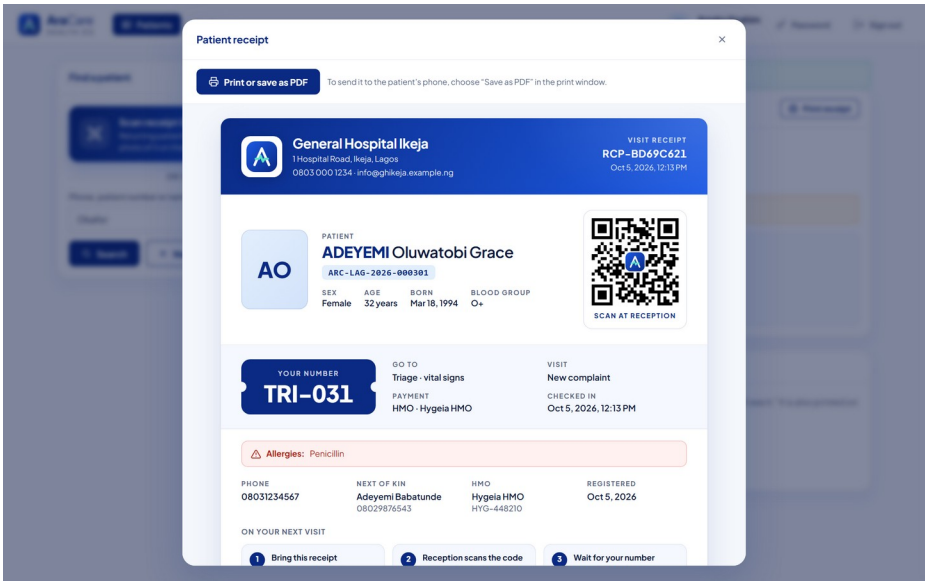


Figure 3.15: *Check-in receipt with QR code and the triage ticket number TRI-031.*

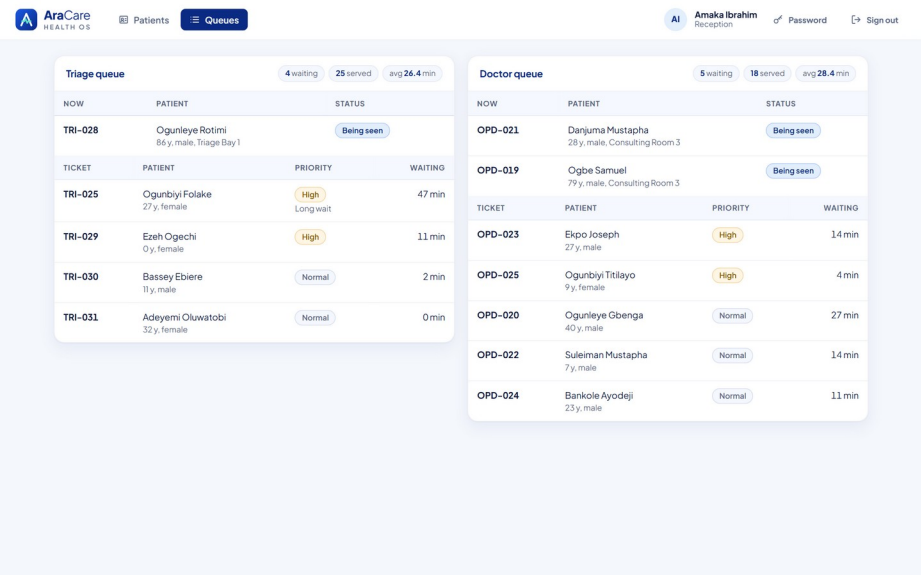


Figure 3.16: *Live queue overview for triage and the doctor queue.*

3.9.4 Nurse Triage

The nurse sees the triage queue in priority order (Figure 3.17, left). TRI-025, a NORMAL patient who has waited 47 minutes, is shown as HIGH with a "Long wait" label: this is the aging rule at work. Clicking "Call next patient" calls the next ticket and opens the vital-signs form (right), where the nurse marks the patient as arrived or not here.

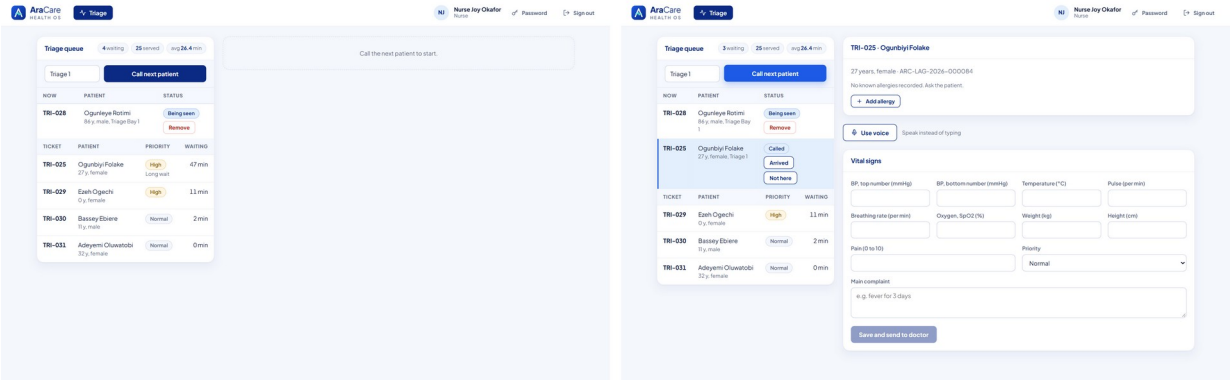


Figure 3.17: *Nurse triage dashboard with TRI-025 promoted by aging (left), and a patient called with the vital-signs form open (right).*

Instead of typing, the nurse can use the voice note. After the patient agrees to be recorded, the nurse records or types what was said, and "Fill in the form" extracts each value into the vital-signs form for the nurse to check (Figure 3.18). On saving, the system flags abnormal values: here fever 38.9 °C, BP 150/95, pulse 108 and respiratory rate 22 are flagged, and the patient moves to the doctor queue (Figure 3.19).

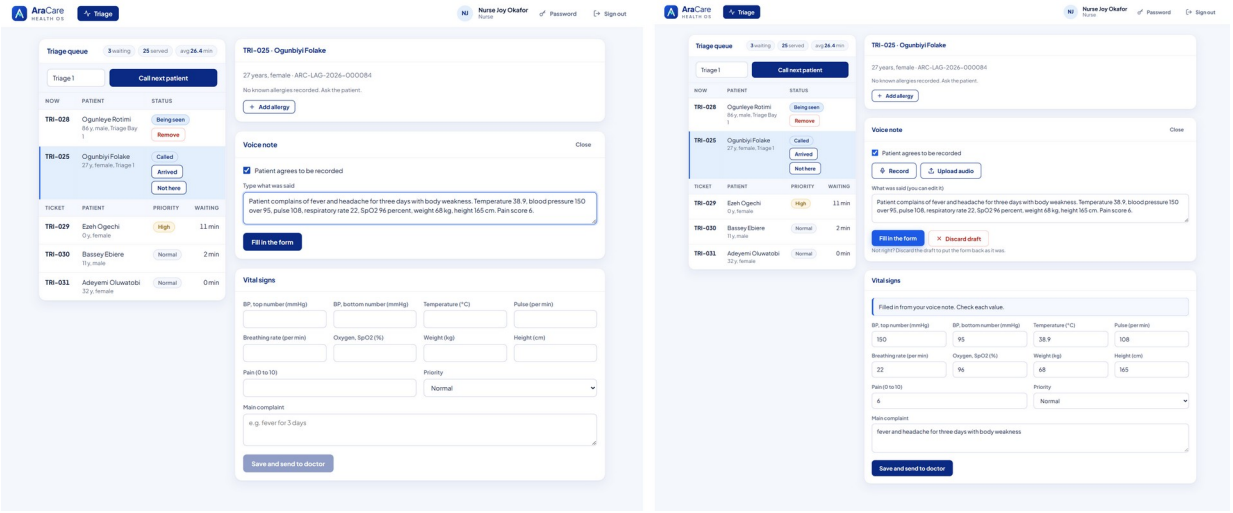


Figure 3.18: *Voice note with recorded consent (left) and the vital signs filled in from it, for the nurse to check (right).*

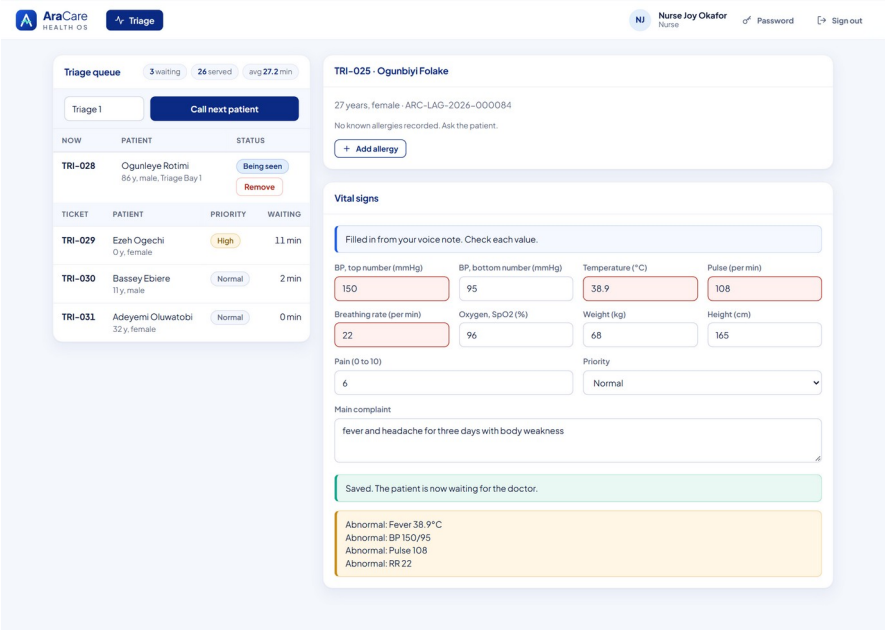


Figure 3.19: *Triage saved, with abnormal vital signs flagged.*

3.9.5 Doctor Consultation

The doctor's queue works the same way, and calling a patient opens their chart with today's vitals and complaint (Figure 3.20). The doctor can start from a template such as "Uncomplicated malaria", which pre-fills the note, diagnosis, tests and drugs, or dictate the consultation and let the scribe draft the SOAP note for review (Figure 3.21).

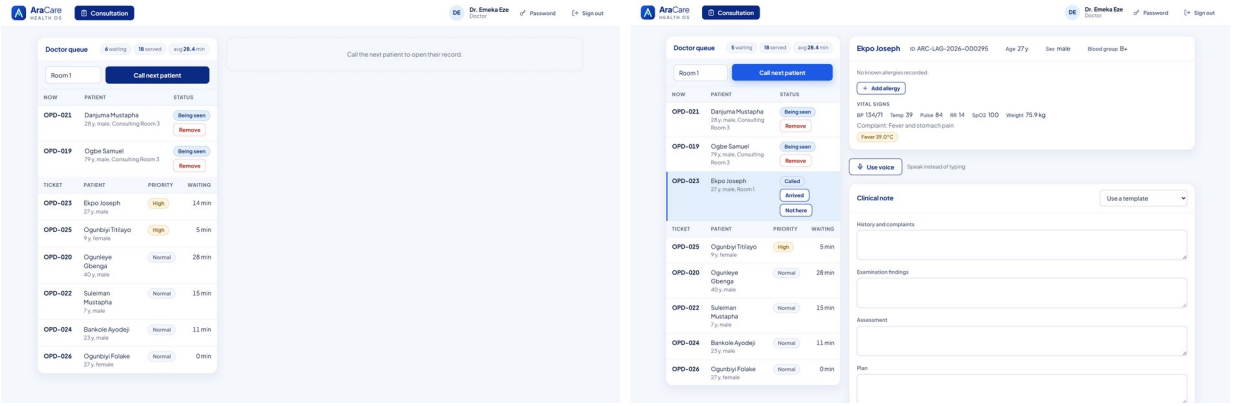


Figure 3.20: *Doctor dashboard with the doctor queue (left) and the patient chart opened for consultation (right).*

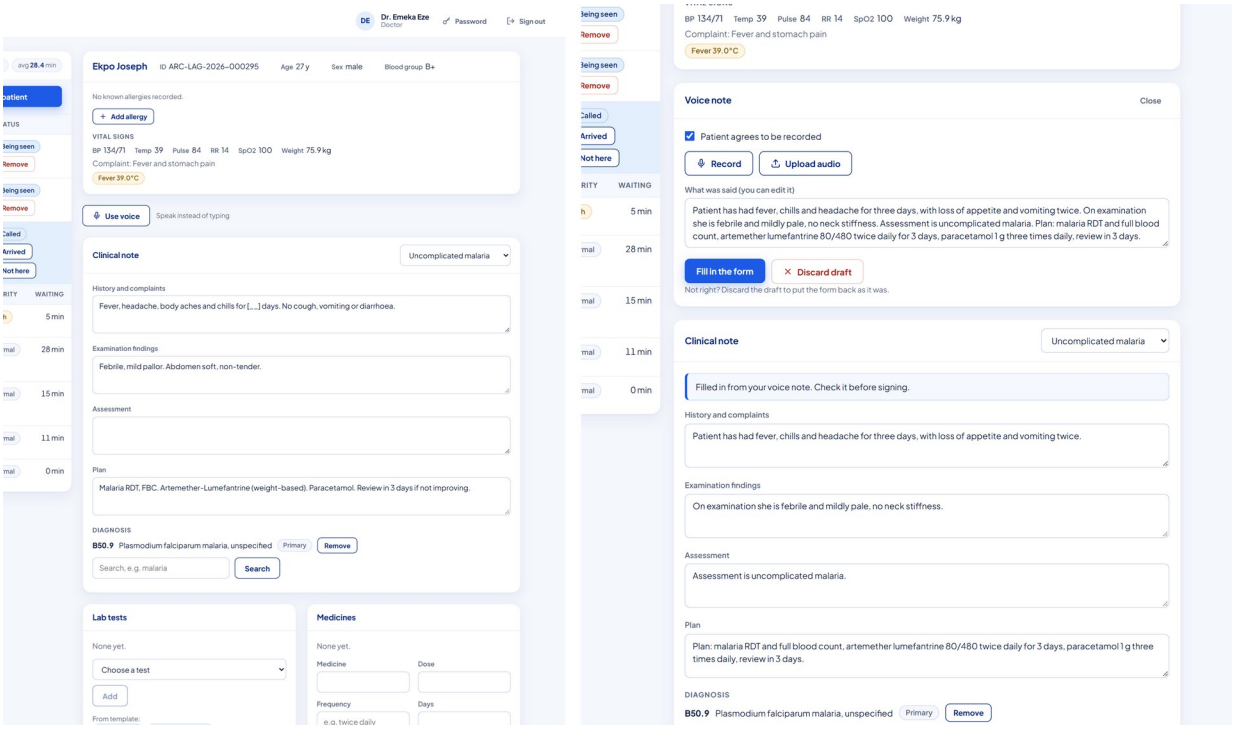


Figure 3.21: *Clinical note from the "Uncomplicated malaria" template (left) and a SOAP note drafted from the doctor's dictation (right).*

Diagnoses are searched by keyword and stored with their ICD-10 code; here "malaria" finds B50.9, Plasmodium falciparum malaria. Laboratory tests and medicines are added from suggestions or typed in (Figure 3.22).

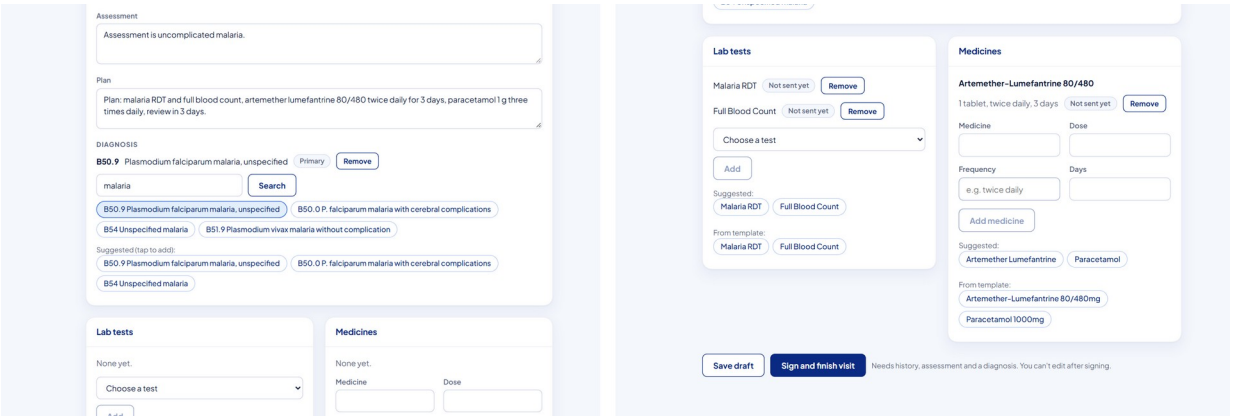


Figure 3.22: *ICD-10 diagnosis search (left) and laboratory orders and prescriptions (right).*

Figure 3.23 shows the allergy hard-block in action. This patient is recorded as allergic to penicillin, and the doctor tries to prescribe amoxicillin. The server refuses it, explains why, and offers to choose another medicine or document a reason to override (left). When the note is signed it becomes read-only, the orders are sent, and the journey tracker shows the patient's next steps, laboratory and pharmacy (right).

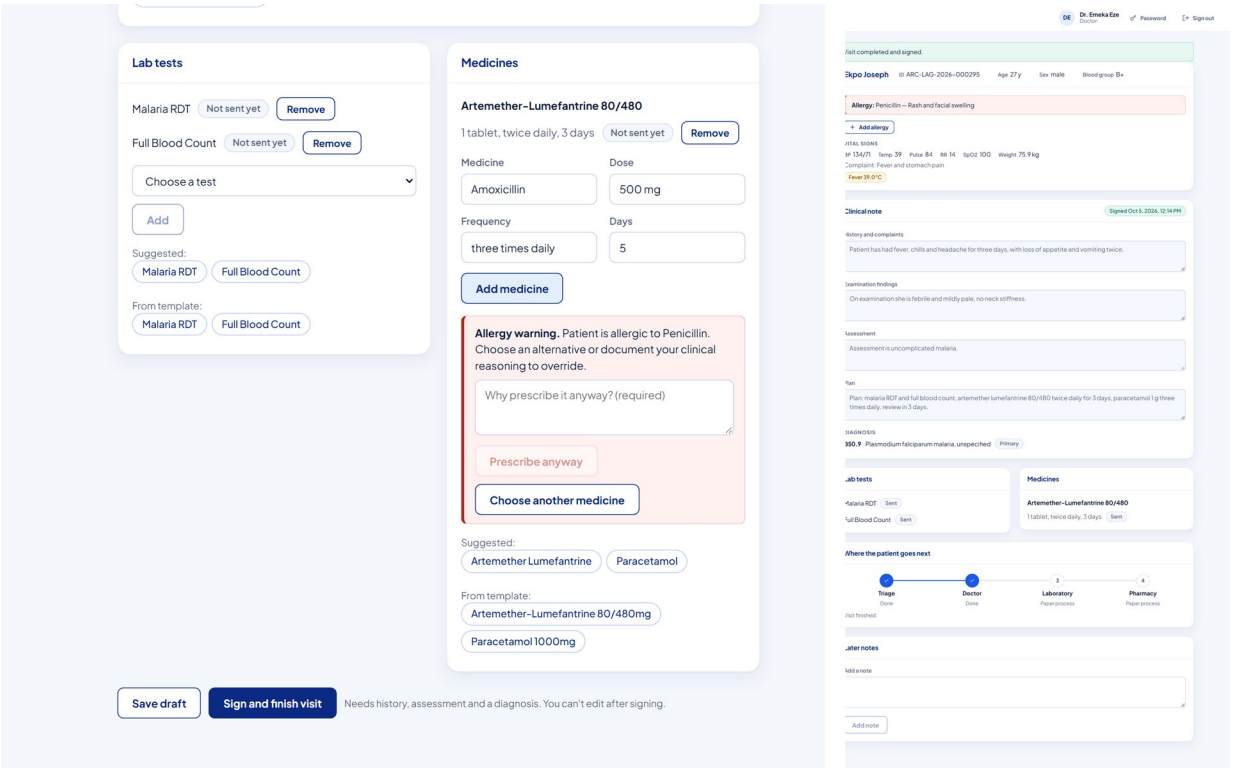


Figure 3.23: *Allergy safety block for a penicillin-allergic patient (left) and the signed visit with the journey tracker (right).*

3.9.6 Administration

Administrators manage staff accounts: they create accounts with a temporary password that must be changed at first login, change roles, reset passwords and deactivate accounts (Figure 3.24). A branding page sets the facility's name, address, logo and receipt motto with a live receipt preview, and visits on the waiver payment track wait for an administrator's approval.

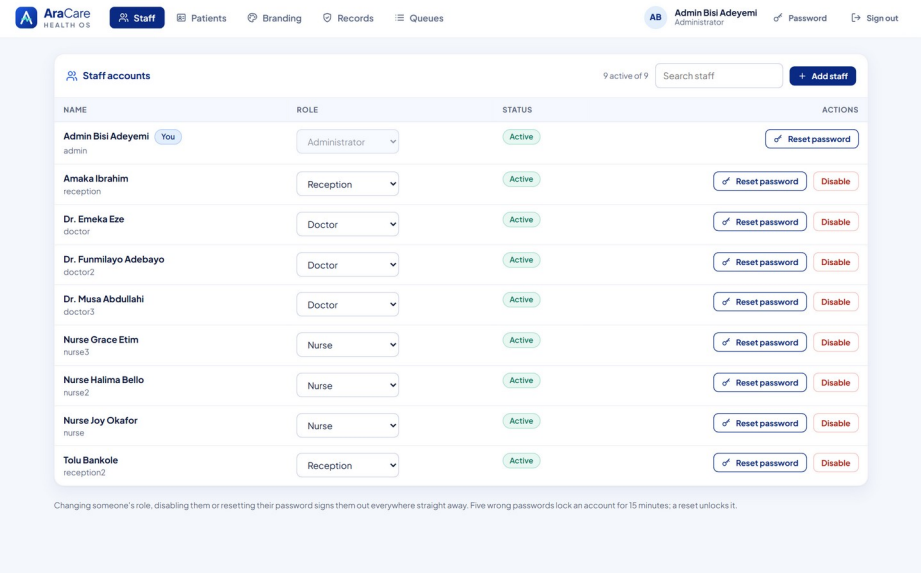


Figure 3.24: *Staff management.*

The records pages give full accountability: the audit log of who did what and when, the system event log from the outbox, and the queued SMS messages to patients.

3.10 DATA ANALYSIS OF ARACARE RECORDS (PYTHON NOTEBOOK)

A hospital system is also a source of data for better decisions. To evaluate whether AraCare's rules work and to show what a manager could learn from the system, I analysed six weeks of records in a Jupyter notebook (`report/notebooks/aracare_patient_flow_analysis.ipynb`), following the data science workflow from Chapter Two. Because real patient data cannot be used in a student report, the dataset was produced by the project's own data generator (`python -m app.mock_data --seed 2026`), which writes backdated history through the same rules as the live code: ticket numbering, the journey cascade, triage flags, the allergy block and signed-note hashing. It contains 407 patients, 852 visits, 1,664 queue tickets, 812 vital-sign records and 777 signed notes from 24 August to 5 October 2026.

Preparation. The tables were loaded from the database with pandas, timestamps were converted from UTC to Lagos time, and ages were computed from dates of birth. Blood pressure was missing in 209 vital-sign records, because it is not routinely taken for young children, so those gaps were left as missing rather than filled with invented values.

Who are the patients?

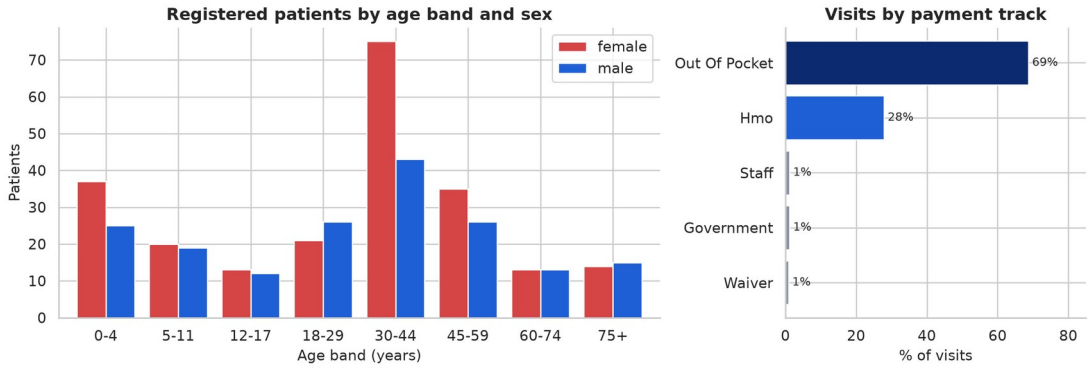


Figure 3.25: *Registered patients by age band and sex, and visits by payment track.*

The largest group is women aged 30 to 44, reflecting antenatal care and typical health-seeking behaviour, and about one in four patients is a child (Figure 3.25). About 69% of visits are paid out of pocket and 28% through an HMO, so the billing module must handle cash and insurance claims equally well.

When do patients arrive?

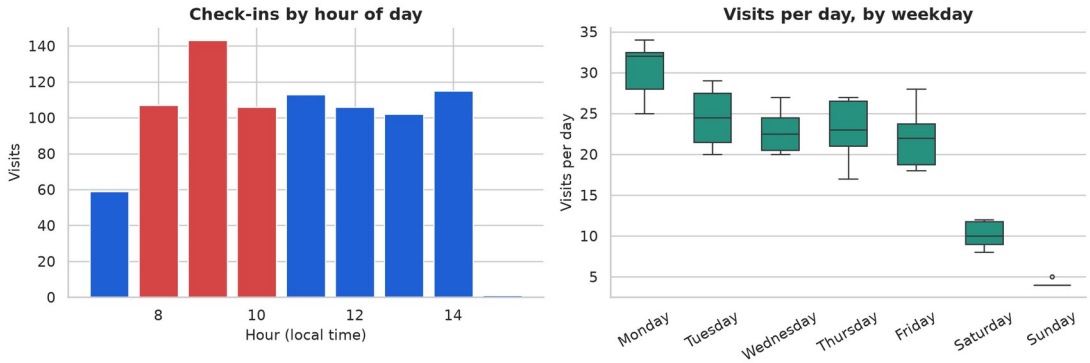


Figure 3.26: *Check-ins by hour of day and visits per day by weekday.*

About 42% of all check-ins happen between 08:00 and 11:00, with the peak at 09:00 (Figure 3.26). Monday is the busiest day, at around 30 visits, weekdays settle at 20 to 25 and weekends are quiet. The 7-day average is flat at about 20 visits per day, so the pattern is a weekly cycle

rather than growth. **Recommendation:** staff an extra triage bay on weekday mornings, especially Mondays.

Does the priority system work?

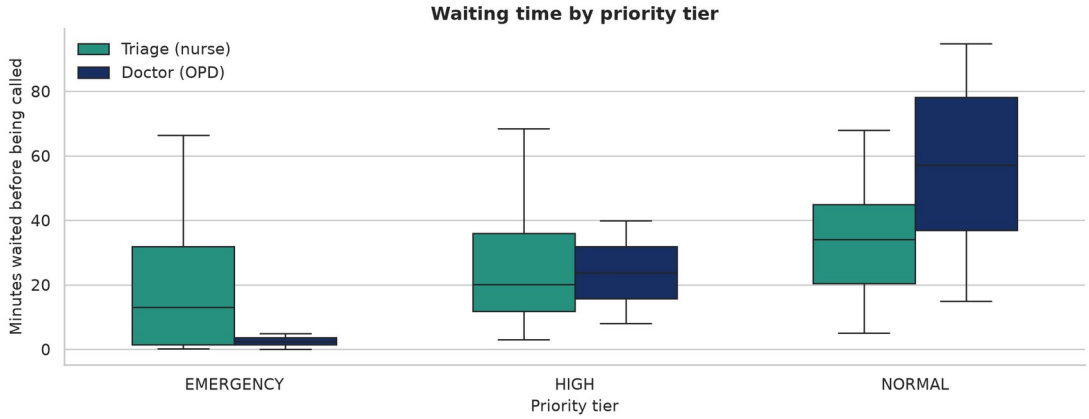


Figure 3.27: *Waiting time before being called, by priority tier and queue.*

Table 3.4: *Waiting time by queue and priority tier.*

Queue	Priority	Patients	Median wait (min)	Maximum wait (min)
Doctor (OPD)	EMERGENCY	34	2.4	4.9
Doctor (OPD)	HIGH	433	23.8	39.9
Doctor (OPD)	NORMAL	309	57.2	94.8
Triage (nurse)	EMERGENCY	34	13.0	66.4
Triage (nurse)	HIGH	456	20.1	68.5
Triage (nurse)	NORMAL	322	34.0	68.0

The doctor queue shows the priority system working exactly as designed (Figure 3.27 and Table 3.4): median waits of about 2 minutes for EMERGENCY, 24 for HIGH and 57 for NORMAL patients. Yet no NORMAL patient waited more than about 95 minutes, which shows that weighted fair queuing and aging prevent starvation. In the triage queue the differences are smaller because priority is only fully known after the nurse has taken the vital signs: the tier shown there is the one the patient ended up with.

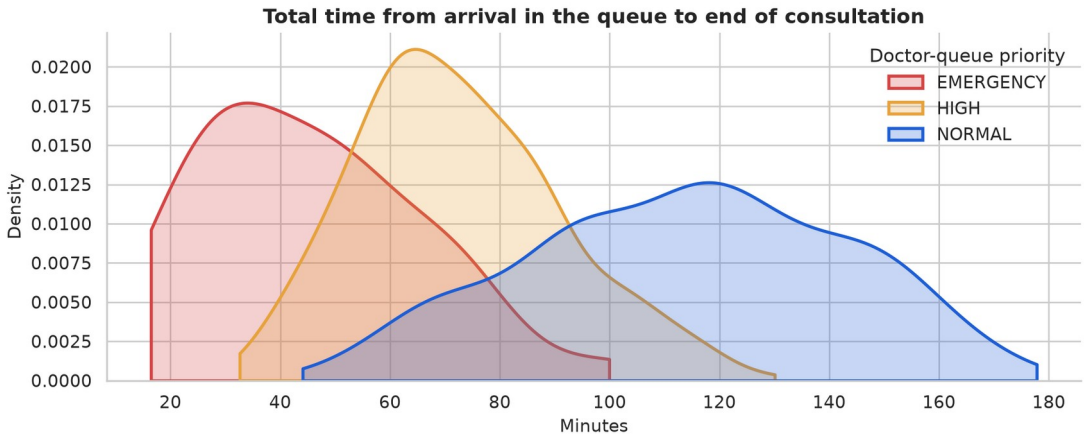


Figure 3.28: *Total time from joining the triage queue to the end of the consultation.*

End to end, a typical NORMAL patient spends about 116 minutes from joining the triage queue to leaving the consulting room, a HIGH-priority patient about 70 minutes and an emergency about 44 minutes (Figure 3.28). Most of that time is waiting, which is why the receipt, SMS and TV display matter: patients can wait outside and return when their number is close.

Clinical picture

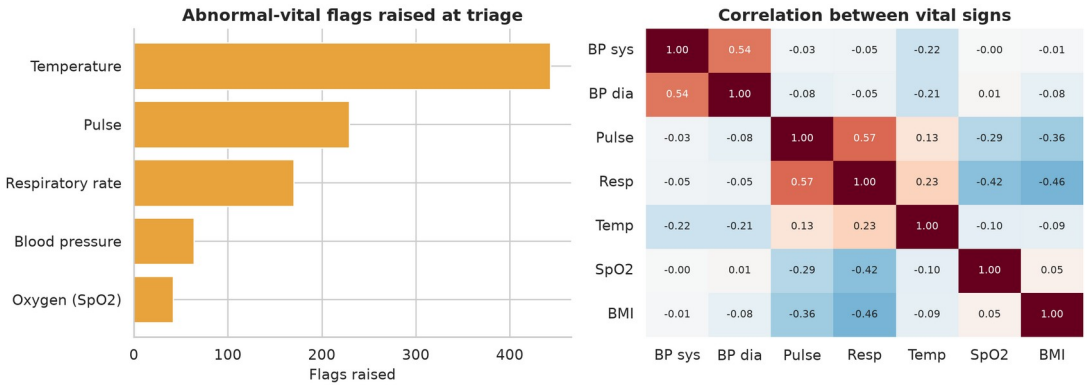


Figure 3.29: *Abnormal-vital flags raised at triage, and correlation between vital signs.*

Three in four triaged patients had at least one abnormal vital sign, and fever was by far the most common flag (Figure 3.29), as expected in a malaria-endemic area. Pulse and respiratory rate rise together (r = 0.57) and both fall as SpO2 rises, which supports combining these flags when raising priority.

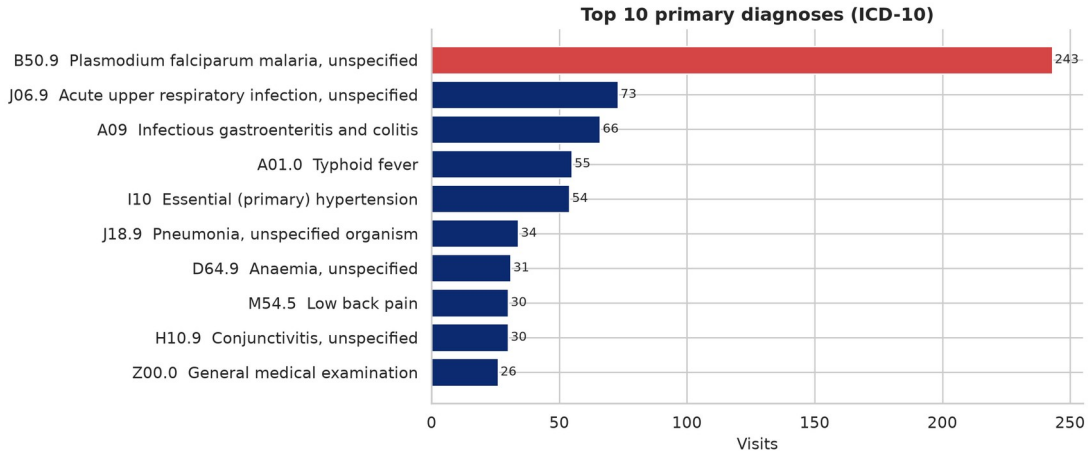


Figure 3.30: *Top 10 primary diagnoses by ICD-10 code.*

Malaria (B50.9) accounts for 31% of all primary diagnoses, followed by upper respiratory infection (9%) and gastroenteritis (8.5%) (Figure 3.30). Because every diagnosis is coded, this report can be produced instantly for disease surveillance or for planning drug stock, which is impossible with paper notes.

Use of the AI voice scribe

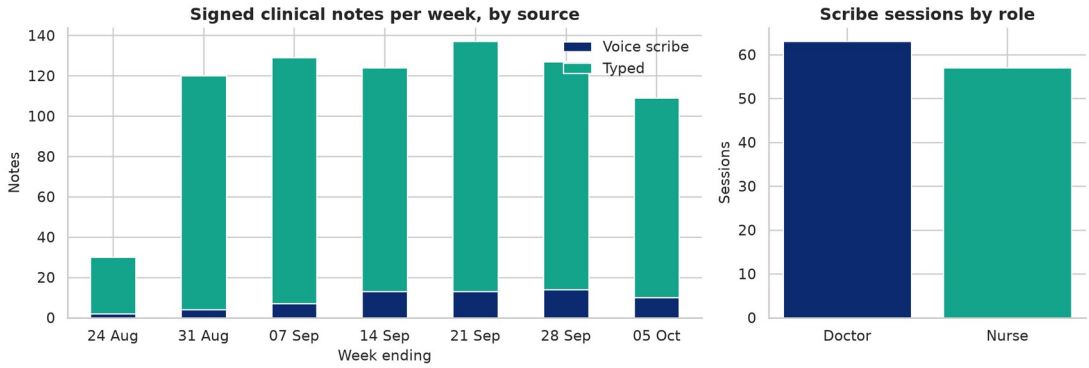


Figure 3.31: *Signed notes per week by source, and scribe sessions by role.*

About 8% of signed notes were drafted by voice, rising from a handful in the first week, and the scribe was used almost equally by doctors (63 sessions) and nurses (57 sessions) (Figure 3.31). Every one of the 120 drafts was reviewed and accepted by a clinician before signing, as the design requires.

Table 3.5: *Summary of findings from the AraCare data analysis.*

Question	Answer from the data
Who are the patients?	Largest group: women aged 30 to 44; about 1 in 4 is a child; 69% pay out of pocket

When do they arrive?	42% between 08:00 and 11:00; Mondays busiest; about 20 visits per day
Does prioritisation work?	Doctor-queue medians: EMERGENCY 2 min, HIGH 24 min, NORMAL 57 min; no starvation
How long is a visit?	About 2 hours for a NORMAL patient, mostly waiting
What is treated most?	Malaria (31%); fever is the most common triage flag
Is the scribe used safely?	8% of notes; all 120 drafts reviewed and accepted by a clinician

3.11 TESTING AND VALIDATION

Testing focused on the rules that protect patients and on full visit flows. The backend has **47 automated tests**, written with pytest and FastAPI's TestClient, each running against a fresh database. All 47 pass (Figure 3.32).

Table 3.6: *Automated test suite of the AraCare backend.*

Test file	Tests	What is verified
test_flow.py	9	Full visit journey; roles; duplicate detection and fuzzy search; sign requires ICD-10; allergy block and documented override; critical vitals force EMERGENCY
test_queue.py	7	Emergency first; elders outrank adults; WFQ prevents starvation; aging; no-show, rejoin and cancel; TV shows no full names and shows ETAs
test_scribe.py	8	Grounding drops invented numbers; consent gate; Ollama output validated; fallback to rules when Ollama or Whisper is down; audio discarded
test_hardening.py	13	Unsafe production settings refused; login lockout; token revocation; concurrent registrations get unique numbers; day-end sweeper; upload limits; audit record IDs
test_features.py	10	Allergy class aliases (sulfa blocks Septrin); scanner noise tolerance; receipts; photo privacy; branding; broken Whisper install degrades safely

terminal - cd backend && pytest -v

```
$ pytest -v
test_features.py::test_allergy_class_aliases PASSED
test_features.py::test_old_allergy_records_still_match_by_name PASSED
test_features.py::test_sulfa_allergy_from_reception_blocks_septrin PASSED
test_features.py::test_broken_whisper_install_degrades_instead_of_crashing PASSED
test_features.py::test_temporary_password_only_opens_password_change PASSED
test_features.py::test_facility_branding_and_logo PASSED
test_features.py::test_parse_code_tolerates_scanner_noise PASSED
test_features.py::test_scan_lookup PASSED
test_features.py::test_receipt_before_and_after_check_in PASSED
test_features.py::test_patient_photo_is_served_only_to_staff PASSED
test_flow.py::test_auth_required_and_roles PASSED
test_flow.py::test_registration_card_and_patient_number PASSED
... 23 more tests ...
test_queue.py::test_no_show_rejoin_and_second_no_show_cancels PASSED
test_queue.py::test_sweeper_times_out_called_tickets PASSED
test_queue.py::test_public_tv_display_has_no_full_names PASSED
test_queue.py::test_tv_shows_estimated_wait_for_each_waiting_patient PASSED
test_scribe.py::test_grounding_drops_numbers_not_in_transcript PASSED
test_scribe.py::test_rules_nurse_extraction PASSED
test_scribe.py::test_consent_gate_then_draft_then_accept_via_note PASSED
test_scribe.py::test_nurse_scribe_drops_hallucinated_vitals_and_doctor_role_is_protected PASSED
test_scribe.py::test_transcribe_degrades_gracefully_without_whisper PASSED
test_scribe.py::test_ollama_output_is_grounded_and_validated PASSED
test_scribe.py::test_ollama_down_falls_back_to_rules PASSED
test_scribe.py::test_transcribe_endpoint_with_stubbed_whisper_discards_audio PASSED
47 passed in 4.60s
```

Figure 3.32: *Test run: all 47 backend tests pass.*

In addition to automated tests, the complete flow was tested manually through the user interface, from registration to a signed note, on a desktop browser, a 1080p TV screen and a phone-sized screen, producing the screenshots in Section 3.9.

3.12 DEPLOYMENT

A facility server runs AraCare with Docker Compose: one command starts PostgreSQL 16 and the API, the database schema is upgraded automatically by Alembic at start-up, and a health check is available at `/api/health`. In production the first administrator is created from an environment variable and must change the password at first login; interactive API documentation is disabled unless explicitly enabled. Only one API process runs per facility because the queue sweeper and WebSocket hub are in-process; the documentation explains how to back up the database daily and how to place the server behind HTTPS.

3.13 CHALLENGES

- **Fairness versus urgency:** a strict priority queue starved NORMAL patients in early tests. Solving it required the WFQ burst limits and aging, and careful tests to prove both properties at once.

- **Concurrency:** two receptionists or two consulting rooms acting at the same moment could receive the same patient number or call the same patient. Row-level locks on the counter and queue-state rows fixed this.
- **AI hallucination:** the local language model sometimes produced plausible vital signs that were never spoken. The grounding rule and Pydantic validation of model output made this safe.
- **Running AI offline on modest hardware:** Whisper and a 7-billion-parameter model are heavy for a hospital PC, so model sizes are configurable and every AI step has a non-AI fallback.
- **Messy real-world input:** Nigerian name spellings, phone formats and allergy descriptions ("sulfa drugs", "NSAIDs") required fuzzy search, phone normalisation and drug-class aliases.
- **Day boundaries:** visits left open overnight blocked the patient's next check-in; a sweeper now closes them as abandoned after local midnight.

3.14 LIMITATIONS AND FUTURE WORK

Limitations. The current release covers outpatient registration, queues, triage and consultation only. SMS messages are queued but not sent, because no SMS gateway is connected. The analysis in Section 3.10 uses generated data, so its numbers illustrate the system's behaviour rather than measure a real hospital. The voice scribe has not yet been evaluated with real Nigerian clinicians, accents and ward noise, and the system supports a single API process per facility.

Future work. The next releases will implement the modules already designed in `docs/`: billing and the cashier queue with HMO claims, pharmacy and inventory with stock reservation on `Prescribed`, the laboratory queue, capacity and finance dashboards, internal communications and IoT bedside monitoring. Other planned work includes connecting a Nigerian SMS gateway, cloud backup and analytics sync from the outbox, Redis pub/sub to run several API processes, a FHIR API for interoperability, and a pilot in a real facility to measure waiting times before and after deployment and to fine-tune the speech model for local accents.

3.15 CONCLUSION

This chapter presented AraCare Health OS, an offline-first hospital operating system that I designed and built by applying the skills acquired during my SIWES at Acadeva. The system registers and checks in patients with QR receipts, manages queues fairly with weighted fair queuing and aging, shows live status on a waiting-room TV, records triage and consultations with automatic safety checks and ICD-10 coding, and offers a local AI voice scribe that can never write to the record without a clinician's review. It is protected by role-based security and audit logging and verified by 47 passing automated tests. The data analysis confirmed that the queue rules behave as designed and showed how the same records can guide staffing and planning. Together, the placement and the project demonstrated the full journey from data to working software: understanding a real problem, designing carefully, building and testing, analysing the results, and communicating them clearly.

REFERENCES

Acadeva Limited. (2026). Acadeva organisation information for SIWES reports. Uyo: Acadeva Limited.

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.

Demers, A., Keshav, S., & Shenker, S. (1989). Analysis and simulation of a fair queueing algorithm. ACM SIGCOMM Computer Communication Review, 19(4), 1-12.

FastAPI. (2026). FastAPI documentation. https://fastapi.tiangolo.com

Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. Annals of Eugenics, 7(2), 179-188.

Google. (2026). Gemini API documentation: structured output. https://ai.google.dev

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras and TensorFlow (2nd ed.). O'Reilly Media. (Source of the California Housing dataset copy used.)

Industrial Training Fund. (2016). Students Industrial Work Experience Scheme (SIWES): Guidelines and operational procedures. Abuja: ITF.

Kaggle / Encyclopedia Titanica. (n.d.). Titanic passenger dataset (seaborn-data copy). https://github.com/mwaskom/seaborn-data

Pace, R. K., & Barry, R. (1997). Sparse spatial autoregressions. Statistics & Probability Letters, 33(3), 291-297.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Prasad, A., & Chandra, S. (2024). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. Computers & Security, 136, 103545.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. Proceedings of the 40th International Conference on Machine Learning (ICML).

Richardson, C. (2018). Microservices patterns: With examples in Java (transactional outbox pattern). Manning.

World Health Organization. (2019). International statistical classification of diseases and related health problems, 10th revision (ICD-10). Geneva: WHO.

World Health Organization. (2023). World malaria report 2023. Geneva: WHO.

APPENDICES

Appendix A: AraCare API Endpoints (all under /api)

Table A.7: *AraCare API surface.*

Area	Endpoints
Authentication	/auth/login, /auth/me, /auth/change-password
Patients	/patients, /patients/search, /patients/{id}/check-in, receipts, photos
Queues	/queues/{dept}, /queues/{dept}/call-next, /tickets/{id}/{arrived | no-show | rejoin | cancel}
TV display (public)	/display/{dept}; WebSocket /ws/queue
EHR	/ehr/visits/{id}/triage, /ehr/visits/{id}/note, /ehr/notes/{id}/sign, ICD-10 search, orders, allergies
Scribe	/scribe/consent, /scribe/transcribe, /scribe/draft
Administration	/admin/audit, /admin/events, /admin/messages, /admin/staff (list, create, update, reset-password)
Health	/health

Appendix B: Reproducing the Notebooks, Figures and This Report

All analysis and figures in this report can be regenerated from the backend repository:

20. `cd backend && pip install -r requirements-dev.txt && pytest` (runs the 47 tests).
21. **`ARACARE_DATABASE_URL=sqlite:**////tmp/aracare_ds.db python -m app.mock_data` (generates the six-week dataset).
22. `ARACARE_DB=/tmp/aracare_ds.db python report/build/make_notebooks.py` (executes both notebooks and saves the charts).
23. `python report/build/make_figures.py` (diagrams and code snippets), then `python report/build/build_report.py` (this document).

The notebooks are `report/notebooks/aracare_patient_flow_analysis.ipynb` and `report/notebooks/beginner_data_science_projects.ipynb`; the datasets for the beginner projects are in `report/data/`, and the UI screenshots are in `pictures/`.